

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

1.2: Grundlagen: Object Constraint Language

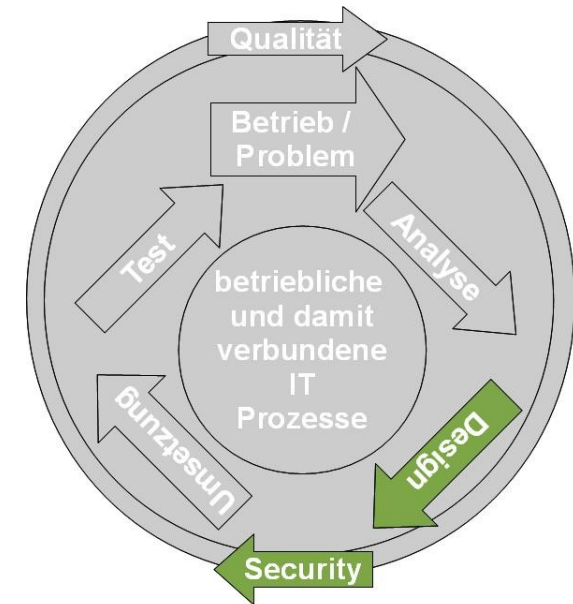
v. 31.10.2016

Einordnung Object Constraint Language (OCL)

Softwarekonstruktion
WS 2016/17



- Modellgetriebene SW-Entwicklung
 - Einführung
 - Modellbasierte Softwareentwicklung
 - Object Constraint Language (OCL)
 - Petrinetze
 - Eclipse Modeling Framework (EMF)
- Qualitätsmanagement
- Testen



Literatur (s. Webseite):

- V. Gruhn: **MDA - Effektives Software-Engineering**. Kapitel 3.6.
- J. Seemann, J.W. Gudenberg: **Software-Entwurf mit UML 2**. Kapitel 14.5.
- Object Management Group: **OCL 2.4** <http://www.omg.org/spec/OCL/2.4/PDF>.

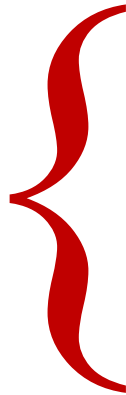
- Entwicklung des Verständnis für die Notwendigkeit der logischen Programmspezifikation
- Grundlegendes Verständnis der Syntax und Ausdrucksmächtigkeit sowie exemplarische Anwendung von OCL
- Entwicklung eines Bewusstseins für die Grenzen der Spezifikation sowie Übersicht über andere Spezifikationsansätze

1.2 Object Constraint Language (OCL)

Softwarekonstruktion
WS 2016/17



1.1 OCL



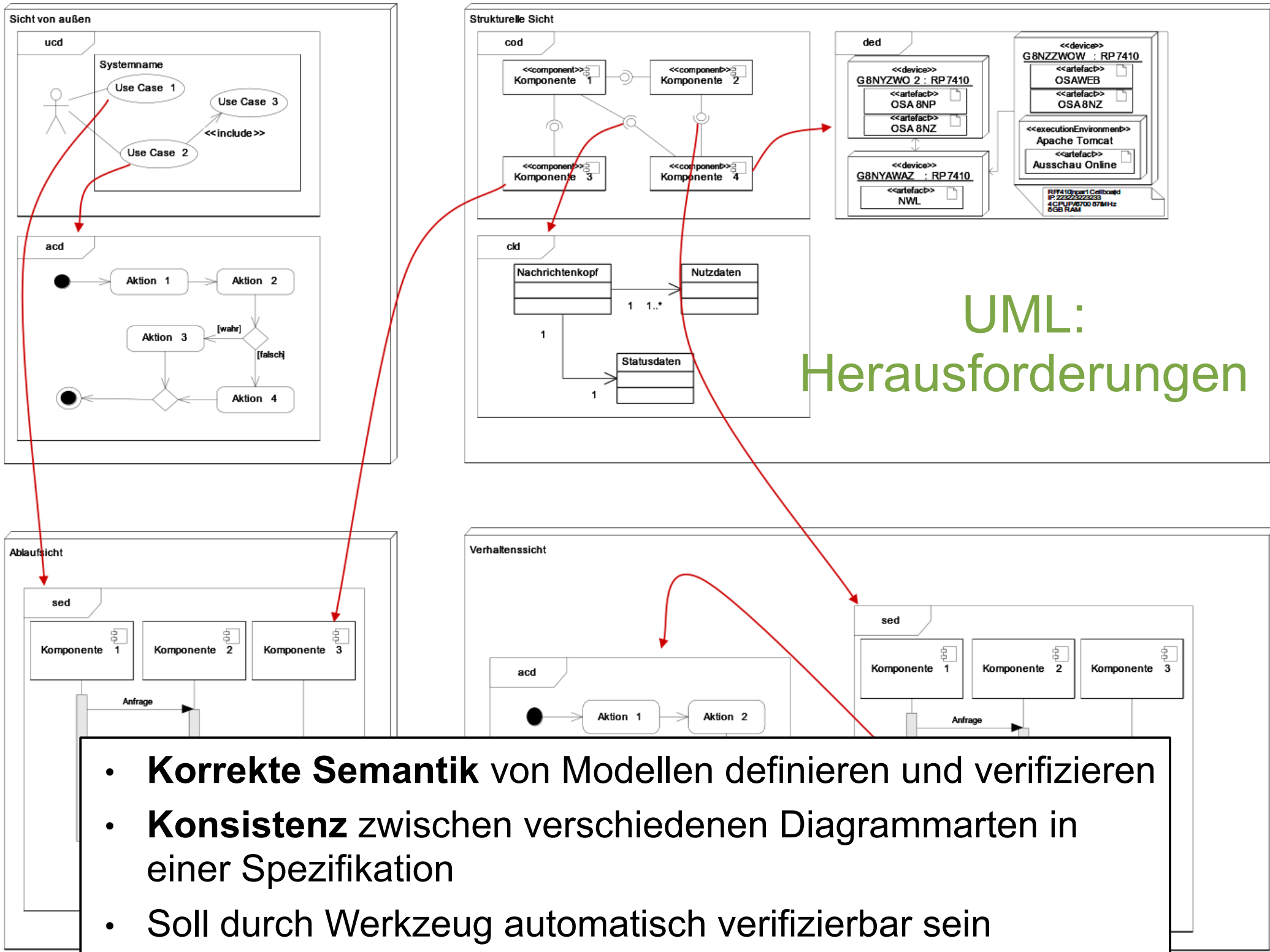
Motivation & Einführung

Assoziationen, Navigationen, Operationen

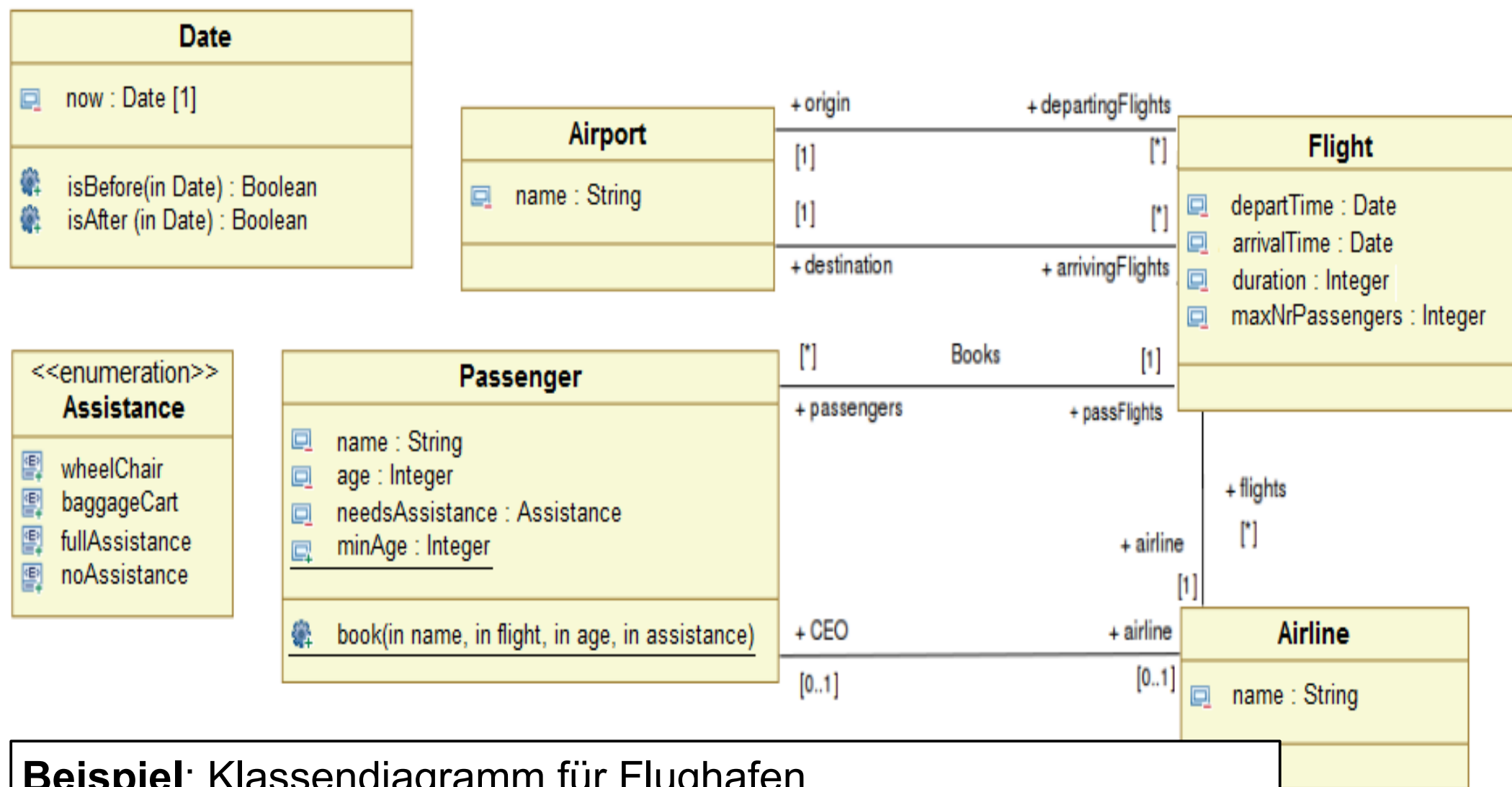
Vor- und Nachbedingungen

Anhang

- **Vorheriger Abschnitt:** Modellbasierte Softwareentwicklung
- **Dieser Abschnitt:** Von Modellen zu Objekten
 - Einführung in Object Constraint Language
→ Syntax und Semantik



UML Herausforderungen: Zur Diskussion



Beispiel: Klassendiagramm für Flughafen.

Bedingung: „Jeder Flug: Abflugdatum gleich/vor Ankunftsdatum.“

Zur Diskussion: Wie in UML-Modell spezifizieren ?

OMG Spezifikation: „**Object Constraint Language 2.0**“.

Teil von UML (<http://www.omg.org/spec/OCL/2.2/PDF>)

Logik-basierte Notation für **Einschränkungen** (Constraints) in UML-Modellen:

- Welche **Klassen erreichbar**; welche Attribute, Operationen und Assoziationen für Objekte dieser Klassen vorhanden.
- **Bedingungen** an Wertebelegungen während Ausführung der modellierten Systemteile (vgl. Assertions in Programm Quellcode).

Unterstützt QS bei modellbasierter Softwareentwicklung.

Vorteile:

- **Automatische Verifikation der Bedingung.**
- **Präzise Spezifikation der Bedingungen beseitigt Mehrdeutigkeit.**
- **Werkzeuge erzeugen aus OCL Assertions in Java.**

Keine Programmiersprache:

→ Modellierung von Programmlogik nicht vorgesehen und möglich !

- Insbesondere: OCL-Ausdrücke **ohne Seiteneffekte**:
 - Kann nichts im Modell verändern.
 - OCL-Ausdruck verursacht keine Zustandsänderung, auch wenn er sie spezifizieren kann (z.B. in einer Nachbedingung).

OCL unterstützt Prüfung von **strenger Typisierung**.

OCL-Ausdrücke: an UML-Modell gebunden.

Beschreiben Einschränkungen für Elemente des Modells, zu dem sie gehören.

Zwei Arten von **Einschränkungen** spezifizieren und verifizieren:

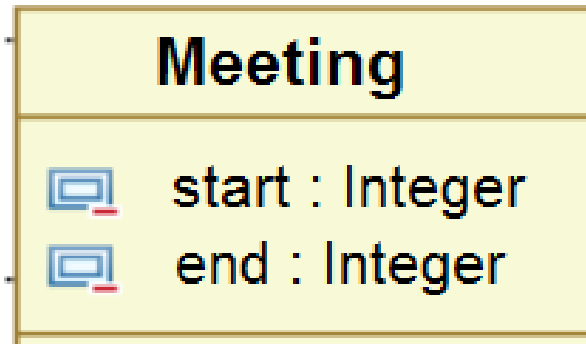
- **Fortlaufende** Zustandsbeschränkung (mit Invarianten).
- Zustandsbeschränkungen **vor** bzw. **nach Methodenaufruf** (mit Vor- und Nachbedingungen).

Syntax:

context <identifier>
<constraintType> [<constraintName>]: <boolean expression>

Wobei:

- | | |
|-----------------------------------|--|
| <i>context</i> | Schlüsselwort |
| <identifier> | Klassen- oder Operationsname. <ul style="list-style-type: none">• Zugehöriges Modellelement markieren.• Kann innerhalb <boolean expression> genutzt werden. |
| <constraintType> | Schlüsselwort <i>inv</i> , <i>pre</i> oder <i>post</i> . |
| <constraintName> | optionaler Name für diesen Constraint. |
| <boolean expression> | boolescher Ausdruck. <ul style="list-style-type: none">• Kann andere Modellelemente referenzieren. |



Das Treffen endet, nachdem es startet.

Beispiel

context Meeting
inv startEndConstraint:
self.end > self.start

context <identifier>
<constraintType> [<constraintName>]:
<boolean expression>

Constraint (Einschränkung):

Einschränkung auf einem oder mehreren Teilen eines UML-Modells.

Constraint (Einschränkung):

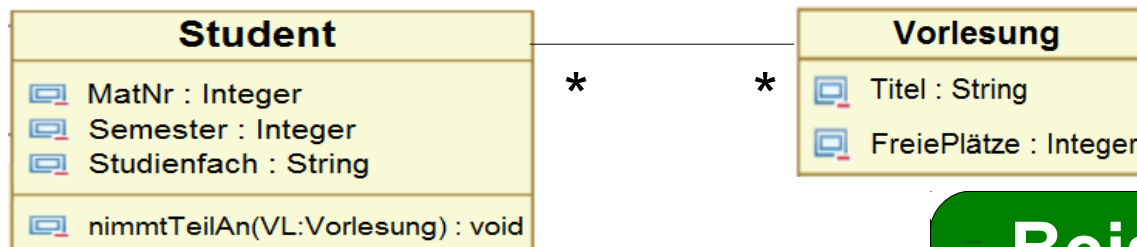
Einschränkung auf einem oder mehreren Teilen eines UML-Modells.

Es gibt die folgenden wichtigsten **Arten** von Einschränkungen:

Class Invariant (Klasseninvariante):

Muss fortwährend von allen Instanzen einer Klasse erfüllt sein

Anhand des Schlüsselwortes *inv* im Kontext der Instanz eines Klassifikators (Klasse, Rollenname...) spezifizierbar



Die Matrikelnummer muss mindestens 5-stellig sein.

Beispiel

context Student

inv: self.MatNr >= 10000

Constraint (Einschränkung):

Einschränkung auf einem oder mehreren Teilen eines UML-Modells.

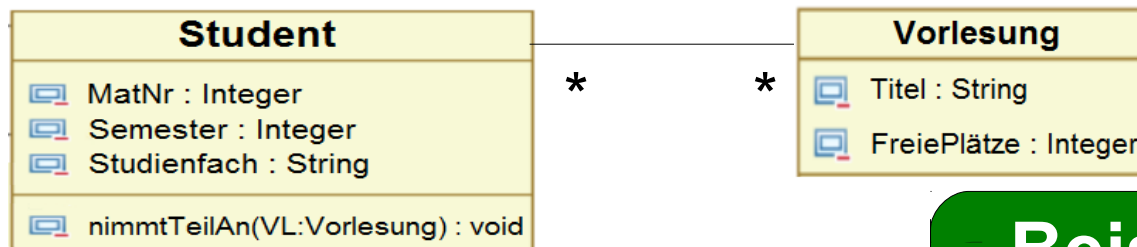
Es gibt die folgenden wichtigsten **Arten** von Einschränkungen:

Class Invariant (Klasseninvariante):

Muss *immer* von allen Instanzen einer Klasse erfüllt sein.

Pre-condition (Vorbedingung):

Muss erfüllt sein, *bevor* Operation ausgeführt wird.



Der Student kann sich nur zu einer Vorlesung anmelden, wenn noch min. ein Platz frei ist.

Beispiel

```
context Student :: nimmtTeilAn (VL:Vorlesung):  
void  
pre: VL.FreiePlätze >= 1
```

Constraint (Einschränkung):

Einschränkung auf einem oder mehreren Teilen eines UML-Modells.

Es gibt die folgenden wichtigsten **Arten** von Einschränkungen:

Class Invariant (Klasseninvariante):

Muss *immer* von allen Instanzen einer Klasse erfüllt sein.

Pre-condition (Vorbedingung):

Muss erfüllt sein, *bevor* Operation ausgeführt wird.

Post-condition (Nachbedingung):

Muss *nach* Ausführen einer Operation erfüllt sein.

Wenn der Student sich zur Vorlesung angemeldet hat, ist ein Platz weniger frei als vorher.

Beispiel

```
context Student :: nimmtTeilAn (VL:Vorlesung):  
    void  
post: VL.FreiePlätze = VL.FreiePlätze @pre -1
```


Folgende **Typen** im OCL-Ausdruck benutzbar:

- **Vordefinierte Typen:**
 - Primitive Typen: String, Integer, Real, Boolean.
 - Kollektions-Typen: Set, Bag, Sequence, OrderedSet.
 - Tupel-Typen: Tuple.
 - Spezielle Typen: OclType, OclAny, ...
- **Klassifikatoren** vom UML-Modell und seinen Eigenschaften:
 - Klassen, Enumerationsklassen und Rollennamen.
 - Attribute und Operationen.

Folgende **Schlüsselwörter** im OCL-Ausdruck benutzbar:

- Konditionalausdrücke: *If-then-else-endif*
- Boolesche Operatoren: *Not, or, and, xor, implies*
- Globale Definitionen: *Def*
- Lokale Definitionen: *Let-in*

Beschränkung von Domänen:

- Beschränkung der Werte, die Attribut annehmen kann.

Beschränkung auf Einmaligkeit:

- Attribut oder Attributmenge einer Klasse für die gilt:
 - Für zwei unterschiedliche Instanzen dieser Klasse dürfen keine gleichen Werte zugewiesen werden.

Regeln für Existenz:

- Bestimmte Objekte / Werte müssen existieren / definiert sein...
- ... bevor andere Objekte / Werte definiert / erzeugt werden können.

- Klasse, die von Invarianten referenziert wird:
Kontext der Invarianten.
- **Gefolgt vom booleschen Wert**, der Invariante angibt.
- Alle Attribute der Kontextklasse in Invarianten nutzbar.

context <identifier>

<constraintType>: <boolean expression>

<identifier> Klassen- oder Operationsname

<constraintType> Schlüsselwort inv, pre, post

<boolean expression> boolescher Ausdruck

- Klasse, die von Invarianten referenziert wird:
Kontext der Invarianten.
- **Gefolgt vom booleschen Wert**, der Invariante angibt.
- Alle Attribute der Kontextklasse in Invarianten nutzbar.
- Beispiel: „Jeder Flug dauert weniger als 4 Stunden.“

Flight	
	departTime : Date
	arrivalTime : Date
	duration : Integer
	maxNrPassengers : Integer

Beispiel

context

context <identifier>

<constraintType>: <boolean expression>

<identifier> Klassen- oder Operationsname

<constraintType> Schlüsselwort inv, pre, post

<boolean expression> boolescher Ausdruck

- Klasse, die von Invarianten referenziert wird:
Kontext der Invarianten.
- **Gefolgt vom booleschen Wert**, der Invariante angibt.
- Alle Attribute der Kontextklasse in Invarianten nutzbar.
- Beispiel: „Jeder Flug dauert weniger als 4 Stunden.“

Flight	
	departTime : Date
	arrivalTime : Date
	duration : Integer
	maxNrPassengers : Integer

Beispiel

context Flight

inv: self.duration < 4

context <identifier>

<constraintType>: <boolean expression>

<identifier> Klassen- oder Operationsname

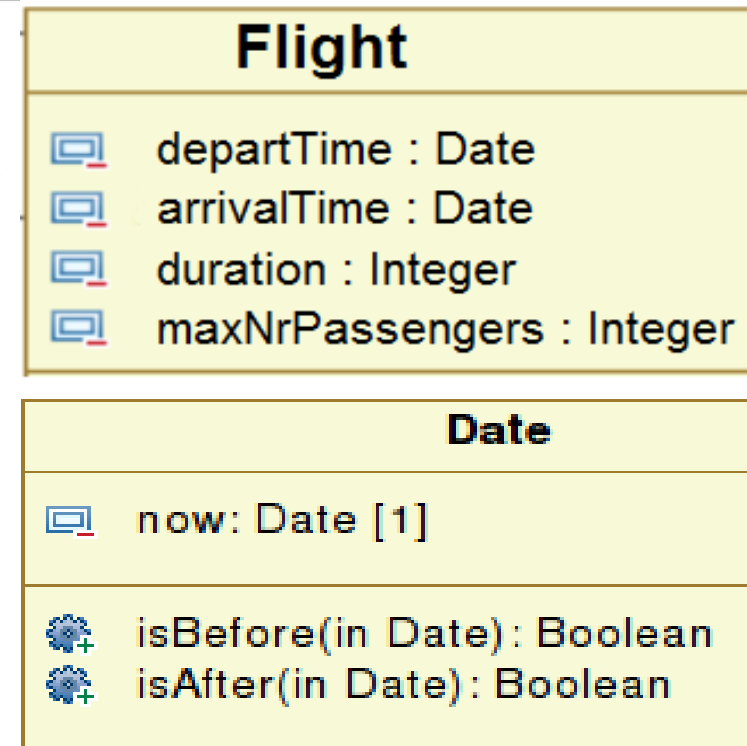
<constraintType> Schlüsselwort inv, pre, post

<boolean expression> boolescher Ausdruck

- Wenn Attributtyp Klasse ist:
 - Attribute und **Abfrageoperationen** dieser Klasse für Erstellung der Invarianten nutzbar (anhand Punkt-Notation).
- Abfrageoperation: Operation, die **Wert von Attributen nicht ändert**.

Invarianten auf Attributen: Abfrageoperationen

- Wenn Attributtyp Klasse ist:
 - Attribute und **Abfrageoperationen** dieser Klasse für Erstellung der Invarianten nutzbar (anhand Punkt-Notation).
- Abfrageoperation: Operation, die **Wert von Attributen nicht ändert**.
- Beispiel: „Abflugdatum eines Fluges ist vor Ankunftsdatum.“



Beispiel

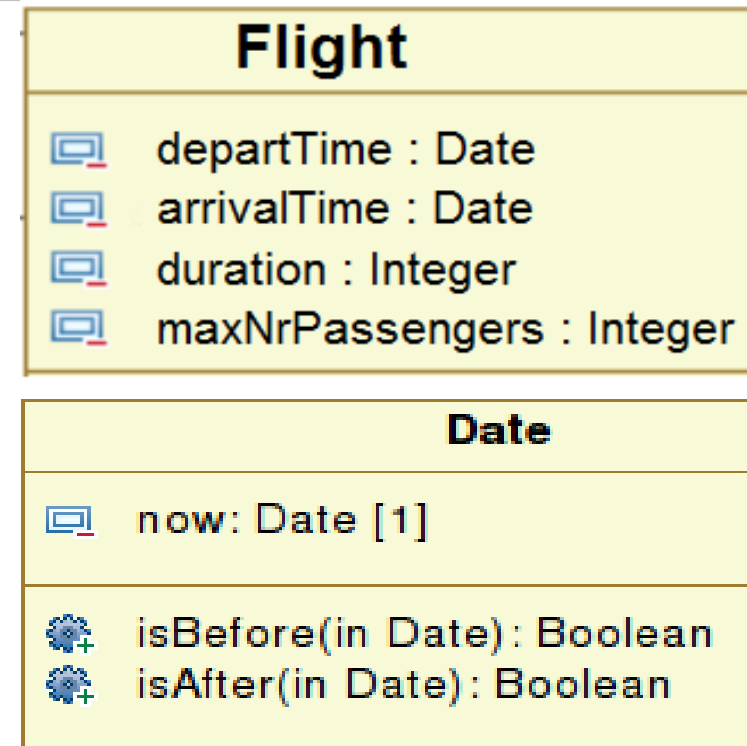
context

context <identifier>
<constraintType>: <boolean expression>

<identifier> Klassen- / Operationsname.
<constraintType> Schlüsselwort inv, pre, post.
<boolean expression> boolescher Ausdruck

Invarianten auf Attributen: Abfrageoperationen

- Wenn Attributtyp Klasse ist:
 - Attribute und **Abfrageoperationen** dieser Klasse für Erstellung der Invarianten nutzbar (anhand Punkt-Notation).
- Abfrageoperation: Operation, die **Wert von Attributen nicht ändert**.
- Beispiel: „Abflugdatum eines Fluges ist vor Ankunftsdatum.“



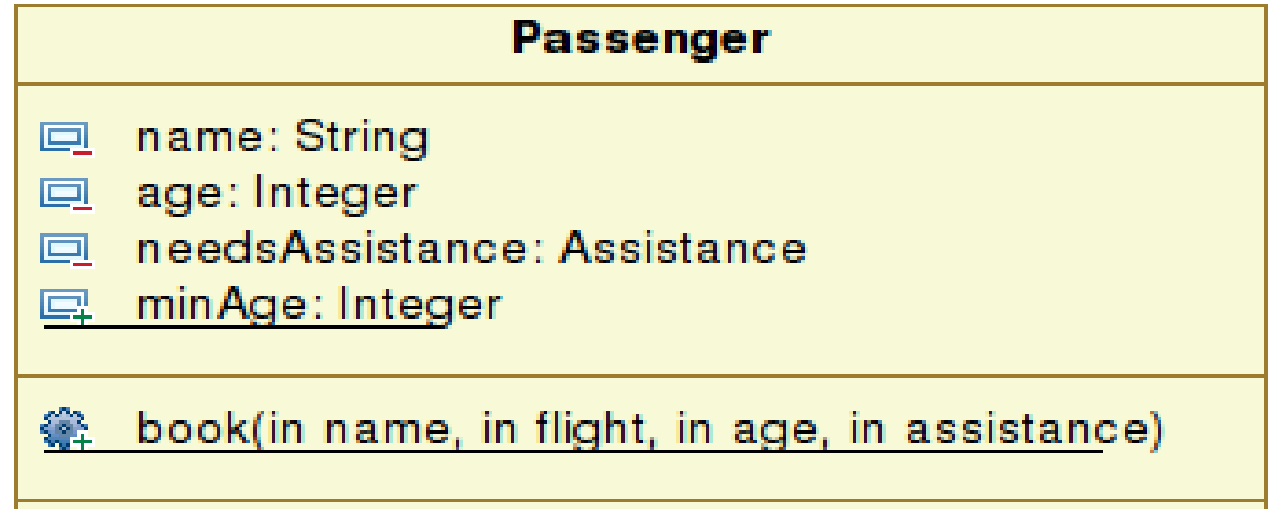
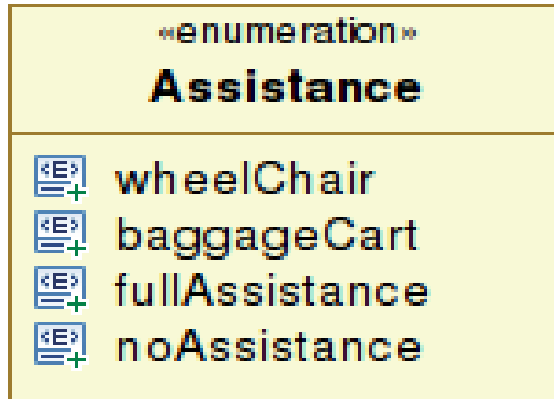
Beispiel

context **Flight**

inv: **self.departTime.isBefore(arrivalTime)**

context <identifier>
<constraintType>: <boolean expression>

<identifier> Klassen- / Operationsname.
<constraintType> Schlüsselwort inv, pre, post.
<boolean expression> boolescher Ausdruck



Aufzählung nutzt Datentypen gefolgt von :: und einem Wert.

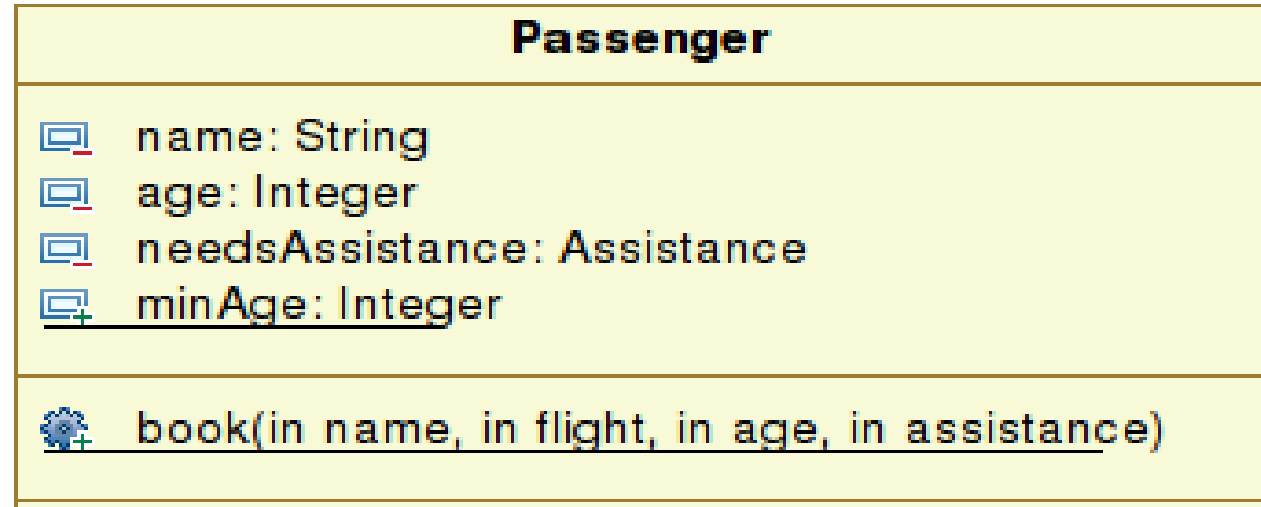
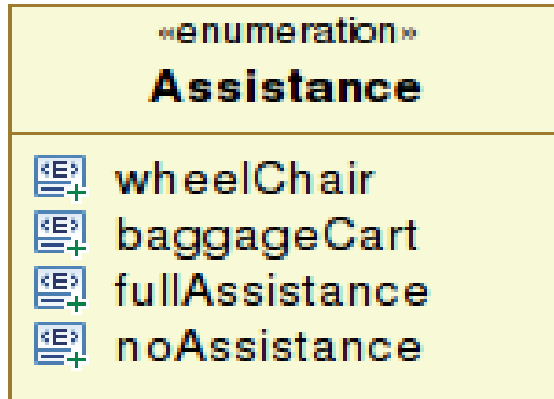
Beispiel

context Passenger

inv: self.age > 95 implies

self.needsAssistance = Assistance :: wheelChair

Bedeutung: ?



Aufzählung nutzt Datentypen gefolgt von :: und einem Wert.

Beispiel

context Passenger

inv: self.age > 95 implies

self.needsAssistance = Assistance :: wheelChair

Bedeutung: **Jeder Passagier über 95 braucht einen Rollstuhl.**

1.1 OCL



Motivation & Einführung

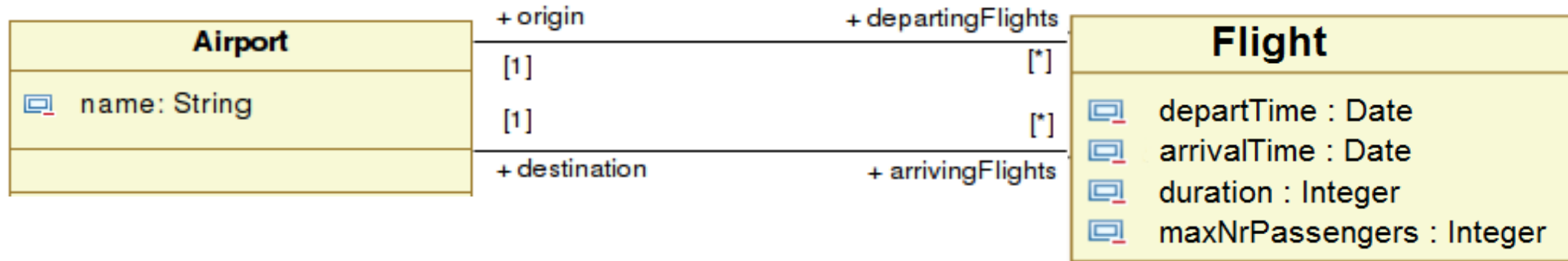
Assoziationen, Navigationen, Operationen

Vor- und Nachbedingungen

Anhang

- Jede Assoziation ist **Navigationspfad**.
- Kontext des Ausdrucks ist **Startpunkt**.
- Rollennamen (oder Assoziationsenden) werden genutzt, um **navigierte Assoziationen** zu identifizieren.

Assoziation und Navigation: Beispiel



Beispiel: „Abflugort eines Fluges ist immer ungleich Ziel.“

Beispiel

context

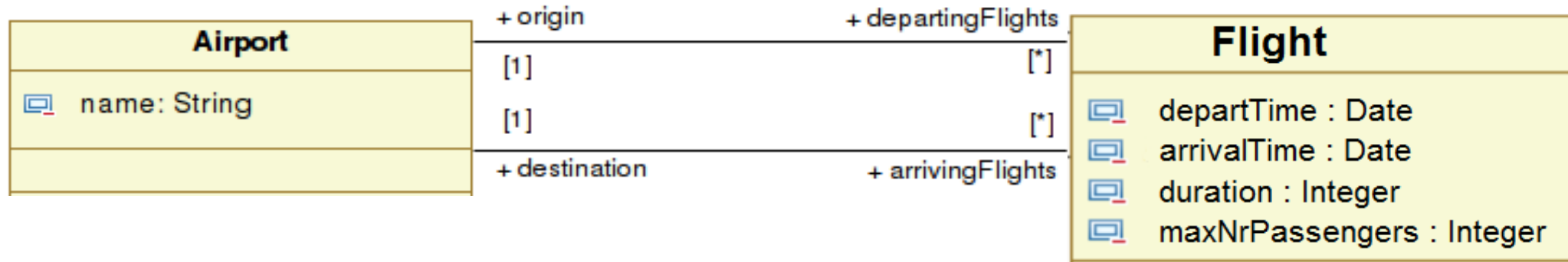
(Annahme: verschiedene Flughäfen haben verschiedene Namen, s. später.)

Beispiel: „Abflugort eines Fluges ist immer Dortmund.“

Beispiel

context

Assoziation und Navigation: Beispiel



Beispiel: „Abflugort eines Fluges ist immer ungleich Ziel.“

Beispiel

context Flight

inv: origin <> destination

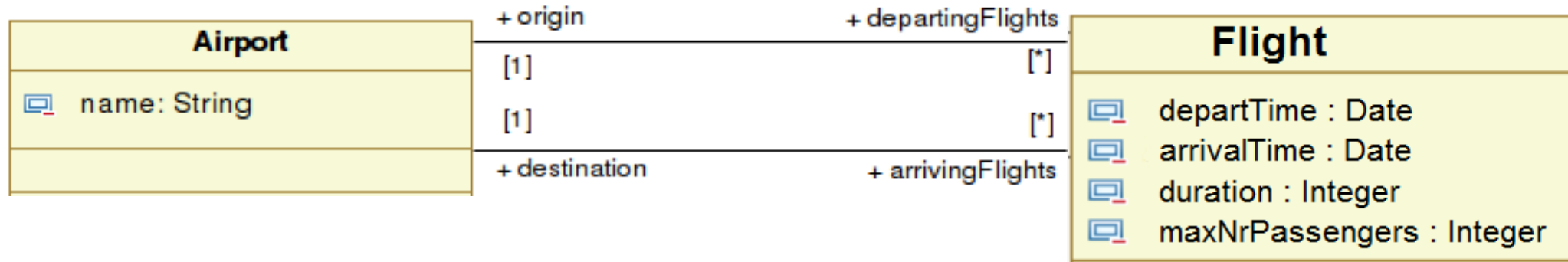
(Annahme: verschiedene Flughäfen haben verschiedene Namen, s. später.)

Beispiel: „Abflugort eines Fluges ist immer Dortmund.“

Beispiel

context

Assoziation und Navigation: Beispiel



Beispiel: „Abflugort eines Fluges ist immer ungleich Ziel.“

Beispiel

context Flight

inv: origin <> destination

(Annahme: verschiedene Flughäfen haben verschiedene Namen, s. später.)

Beispiel: „Abflugort eines Fluges ist immer Dortmund.“

Beispiel

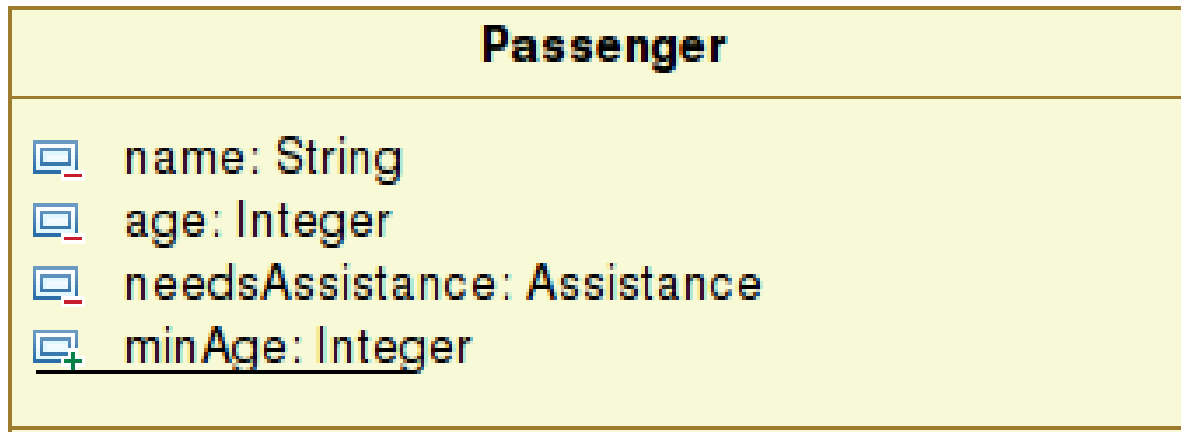
context Flight

inv: origin.name = 'Dortmund'

- Assoziationen: Insbesondere **one-to-many** oder **many-to-many** Beziehungen.
=> Navigation zu Assoziationsende resultiert in Collections.
- OCL Ausdrücke geben entweder Fakt über **alle** Objekte in Collection an oder über Collection **selbst**.
- Beschränkungen beziehen sich oft nur auf Teil einer Collection.
=> Mit Collection-Operationen auswählen:
 - **Pfeil** ($\rightarrow$) spezifiziert Nutzung vordefinierter Operation für Collections. Z.B. für die Collection *passengers* und Größe einer Collection *size()*: *passengers* $\rightarrow$ *size()*.
- Zur **Abgrenzung**: Verwendung von Operation aus UML-Modell (z.B. *departTime.isBefore(arrivalTime)*).

Collection-Operationen: *collect()*

collect(): Operation, um **Attributwerte** zu **sammeln**, z.B.:
`self.passengers` → *collect(name)*.



[*] Books
+ passengers

Ergebnis von
collect() ist eine
Multimenge/Bag!

Bedeutung (in Pseudocode)

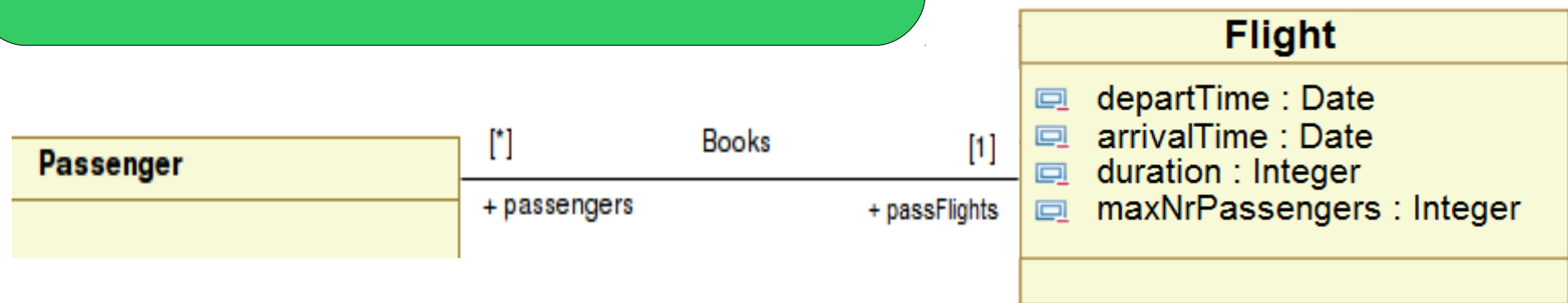
```
Collection<String> c = new Collection();  
foreach (p: passengers) {c.add(p.name);}  
return c;
```

size() spezifiziert die Größe von *Collections*.

Beispiel: Anzahl der Passagiere eines Fluges ist kleiner oder gleich der vorgegebenen maximalen Anzahl der Passagiere.

Beispiel

context



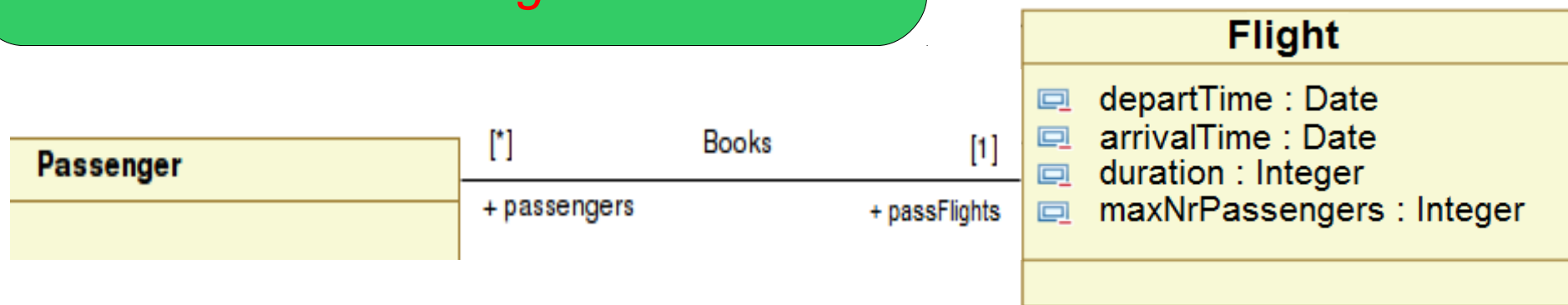
size() spezifiziert die Größe von *Collections*.

Beispiel: Anzahl der Passagiere eines Fluges ist kleiner oder gleich der vorgegebenen maximalen Anzahl der Passagiere.

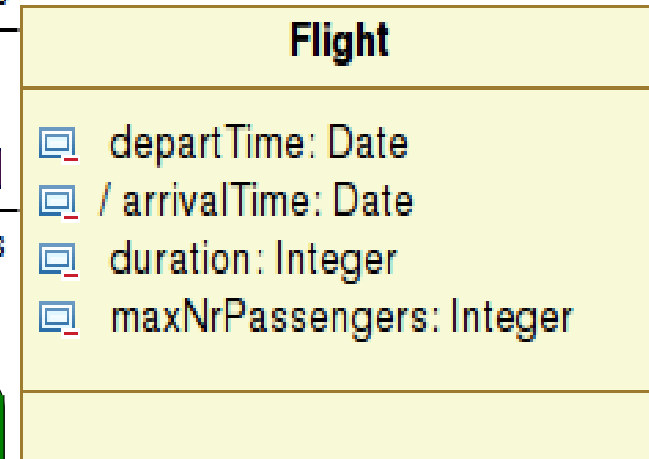
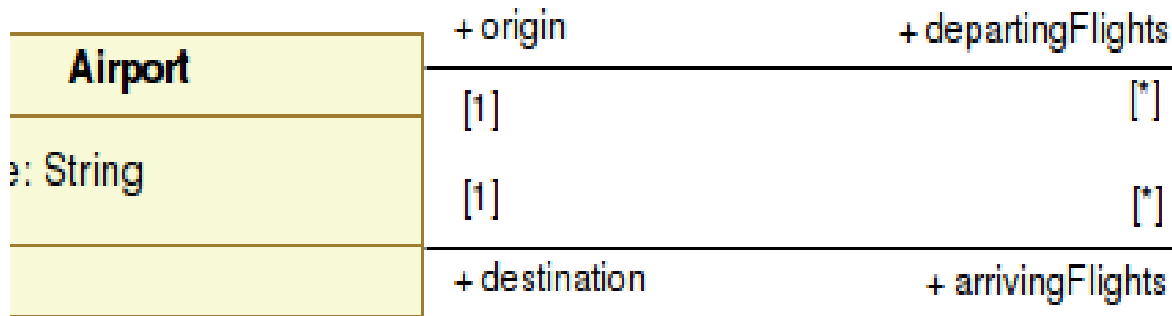
Beispiel

context Flight

*inv: self.passengers → size()
<= self.maxNrPassengers*



Collection-Operationen: *collect()* und *size()*

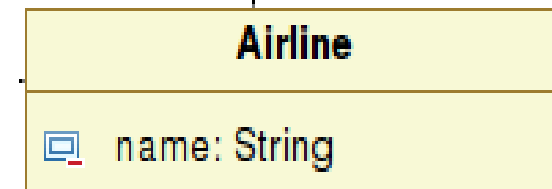
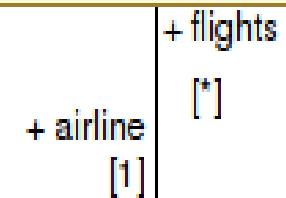


Beispiel

context Airport

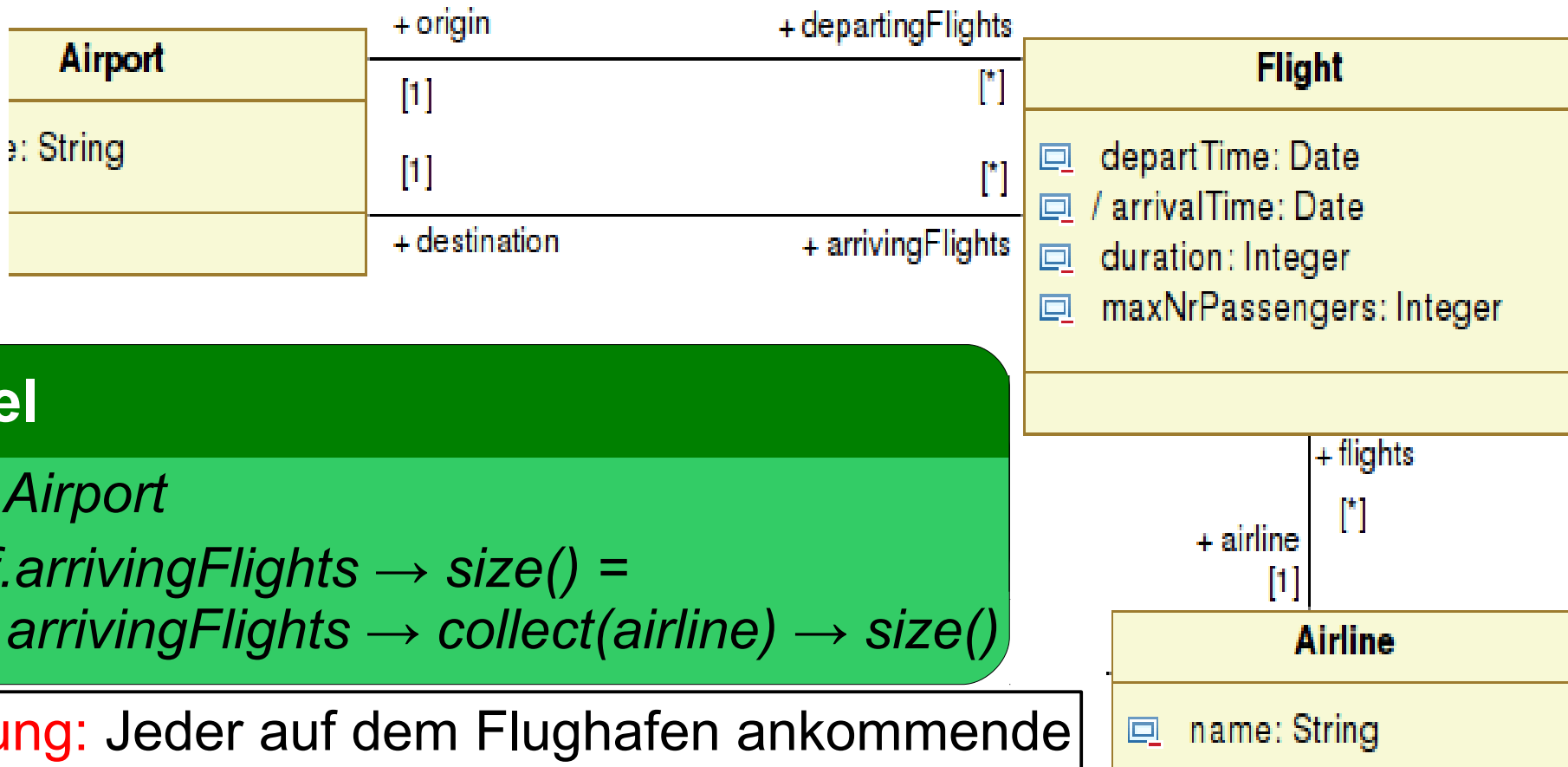
inv: self.arrivingFlights → size() =

self.arrivingFlights → collect(airline) → size()



Bedeutung: ?

Collection-Operationen: *collect()* und *size()*



Beispiel

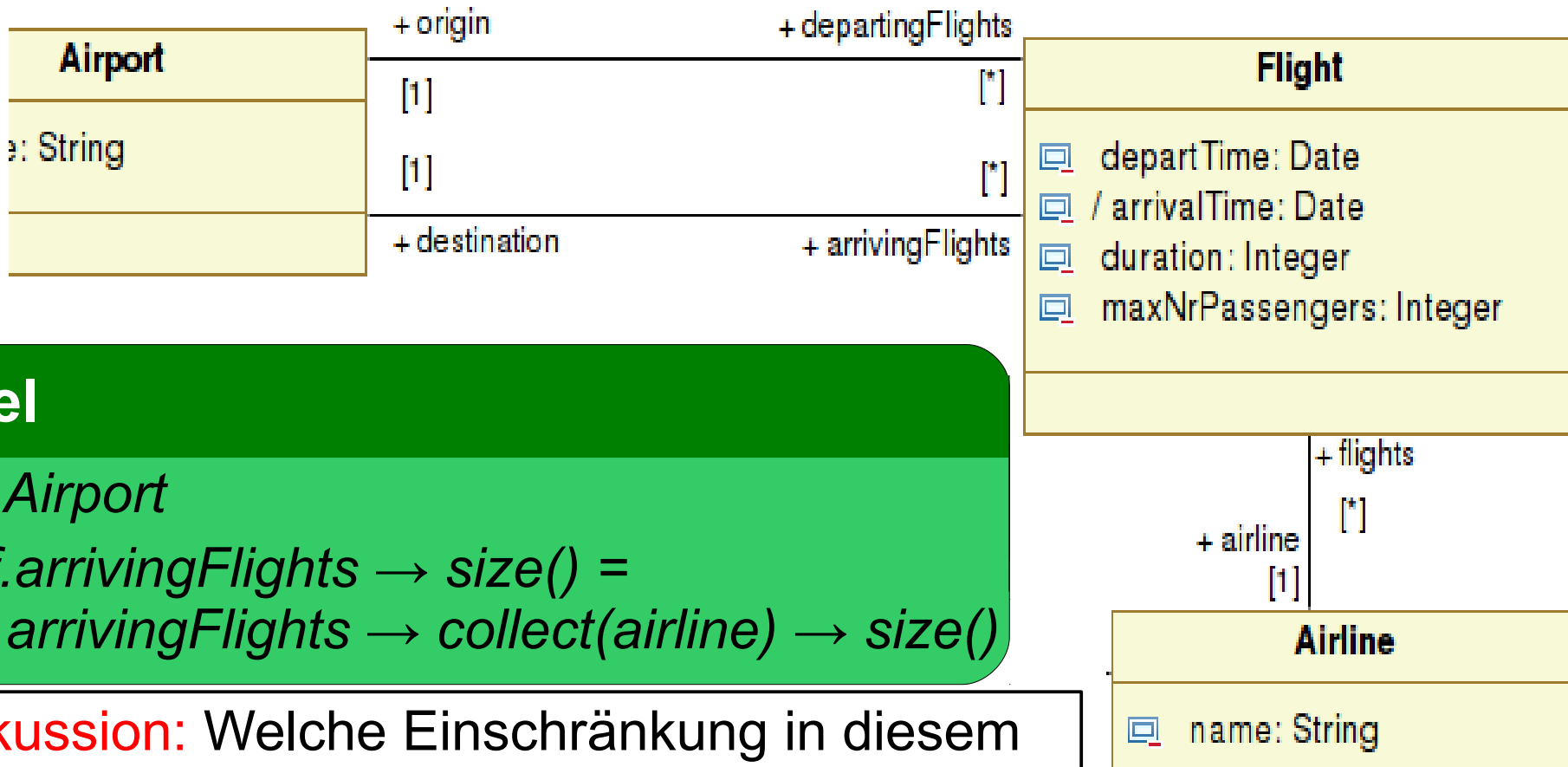
context Airport

*inv: self.arrivingFlights → size() =
self.arrivingFlights → collect(airline) → size()*

Bedeutung: Jeder auf dem Flughafen ankommende Flug gehört zu einer Fluggesellschaft.

Zur Erinnerung: *collect()* erzeugt Multimengen !

Collection-Operationen: *collect()* und *size()*



Beispiel

context Airport

*inv: self.arrivingFlights → size() =
self.arrivingFlights → collect(airline) → size()*

Zur Diskussion: Welche Einschränkung in diesem Klassendiagramm ist besonders wichtig, damit dieser OCL- Ausdruck die erwünschte Bedeutung hat ?

Collection-Operationen: *collect()* und *size()*

Airport	+ origin	+ departingFlights
	[1]	[*]
name: String	[1]	[*]
	+ destination	+ arrivingFlights

Flight
✏ departTime: Date ✏ / arrivalTime: Date ✏ duration: Integer ✏ maxNrPassengers: Integer

Beispiel

context Airport

*inv: self.arrivingFlights → size() =
self.arrivingFlights → collect(airline) → size()*

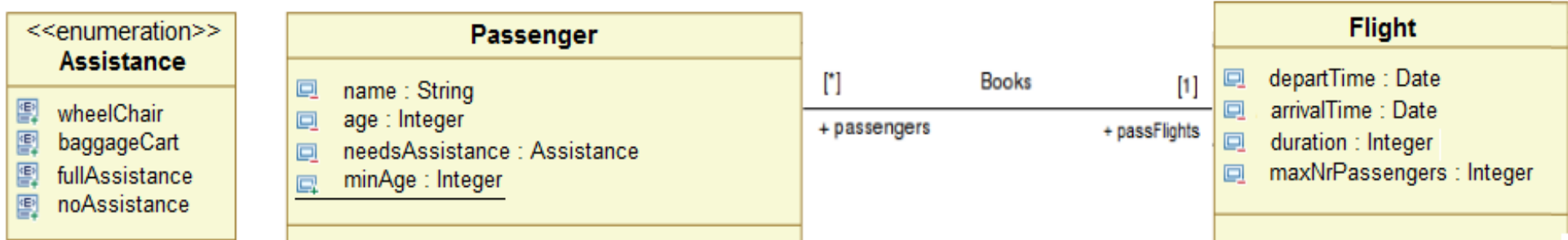
	+ flights
	[*]
+ airline	[1]

Airline
✏ name: String

Antwort: Die Einschränkung, dass jeder Flug
höchstens eine Fluggesellschaft hat.

- Bekommt OCL-Ausdruck als Parameter übergeben.
- **Ergebnis: Subcollection** der verwendeten Collection.
- Liefert alle Elemente einer Collection, für die der Ausdruck **wahr** ist.

- Bekommt OCL-Ausdruck als Parameter übergeben.
- **Ergebnis: Subcollection** der verwendeten Collection.
- Liefert alle Elemente einer Collection, für die der Ausdruck **wahr** ist.



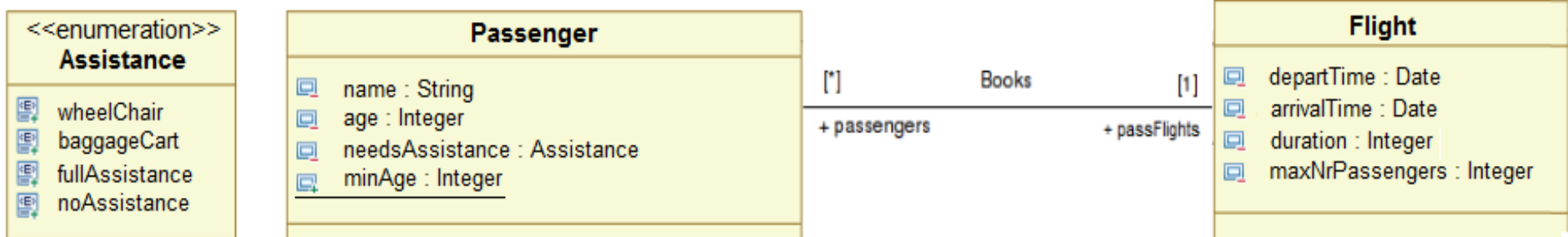
Beispiel

context Flight

*inv: self.passengers → select(needsAssistance
<> Assistance::noAssistance) → size() <= 10*

Bedeutung: ?

- Bekommt OCL-Ausdruck als Parameter übergeben.
- **Ergebnis: Subcollection** der verwendeten Collection.
- Liefert alle Elemente einer Collection, für die der Ausdruck **wahr** ist.



Beispiel

context Flight

*inv: self.passengers → select(needsAssistance
<> Assistance::noAssistance) → size() <= 10*

Bedeutung: Die Anzahl der Passagiere eines Fluges, die Hilfe brauchen, ist kleiner oder gleich 10.

- Verhält sich mengentheoretisch komplementär zu *select()*.
- Liefert alle Elemente einer Collection, für die der Ausdruck **falsch** ist.

Beispiel

context Flight

inv: passengers → select(needsAssistance <> Assistance::noAssistance) → size() <= 10

- Beispiel: Anzahl der Passagiere, die Hilfe brauchen, ist kleiner oder gleich 10. Wie mit *reject()* spezifizieren ?

Beispiel

context Flight

- Verhält sich mengentheoretisch komplementär zu *select()*.
- Liefert alle Elemente einer Collection, für die der Ausdruck **falsch** ist.

Beispiel

context Flight

inv: passengers → ***select***(*needsAssistance* <> *Assistance::noAssistance*) → *size()* <= 10

- Beispiel: Anzahl der Passagiere, die Hilfe brauchen, ist kleiner oder gleich 10. Wie mit *reject()* spezifizieren ?

Beispiel

context Flight

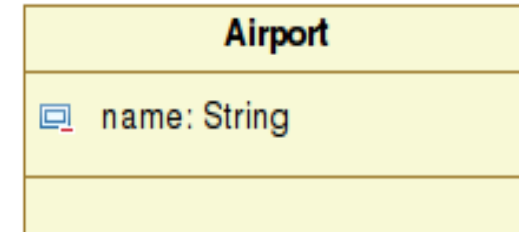
inv: passengers → ***reject***(*needsAssistance* = *Assistance::noAssistance*) → *size()* <= 10

- Nutzbar, um **Bedingung** zu **definieren**.
 - Muss von allen Elementen in Collection eingehalten werden.
- **Erhält OCL-Ausdruck** als Parameter.
- **Liefert booleschen Wert** zurück:
 - Wahr, wenn Bedingung von allen Elementen erfüllt wird.
 - Falsch, sonst.

Beispiel

context Airport

*inv: Airport.allInstances() $\rightarrow$ forAll (a1, a2 |
a1 $\neq$ a2 implies a1.name $\neq$ a2.name)*



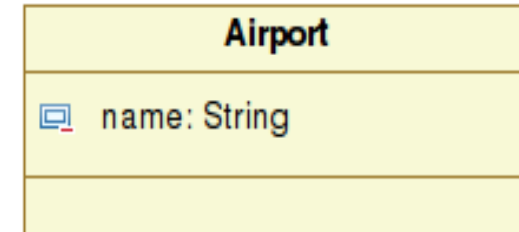
class.allInstances(): Collection mit allen Instanzen einer Klasse.

Bedeutung OCL-Ausdruck: ?

Beispiel

context Airport

*inv: Airport.allInstances() $\rightarrow$ forall (a1, a2 |
a1 $\neq$ a2 implies a1.name $\neq$ a2.name)*



class.allInstances(): Collection mit allen Instanzen einer Klasse.

Bedeutung OCL-Ausdruck: **Jeder Flughafenname ist einzigartig.**

Äquivalent mit Operation *isUnique()*:

context Airport

inv : Airport.allInstances() $\rightarrow$ isUnique(name)

1.1 OCL



Motivation & Einführung

Assoziationen, Navigationen, Operationen

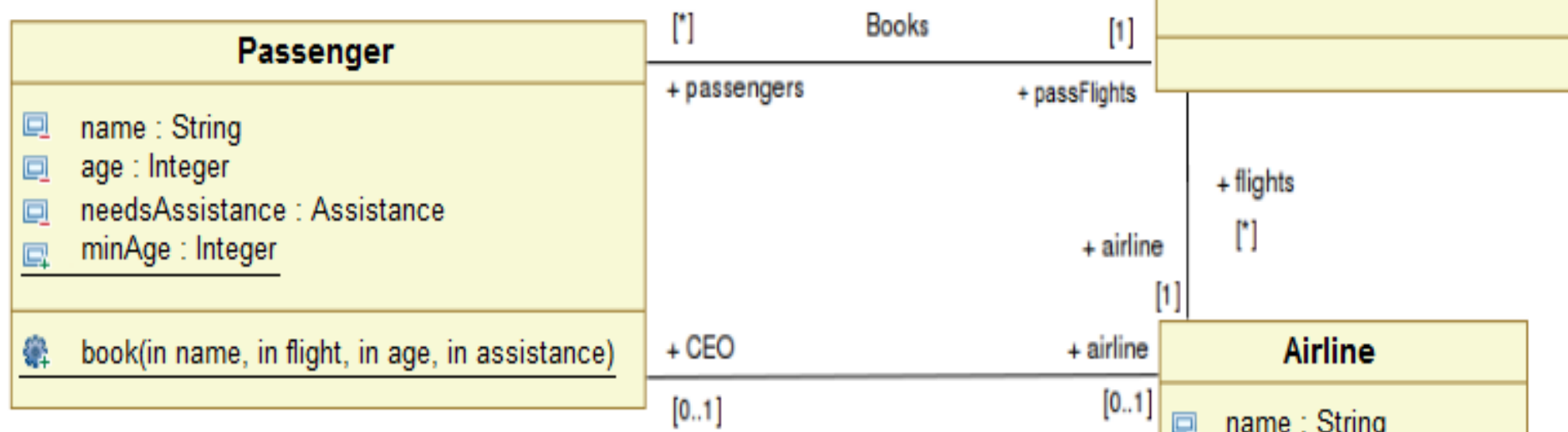
Vor- und Nachbedingungen

Anhang

- In Klassendiagrammen nur Syntax und Signatur einer Operation definierbar.
- **Semantik** einer Operation mittels **Vor- und Nachbedingungen** in OCL spezifizierbar.
- Vorbedingung:
 - Bedingungen von Argumenten und dem initialen Objektzustand müssen erfüllt werden.

Beispiel

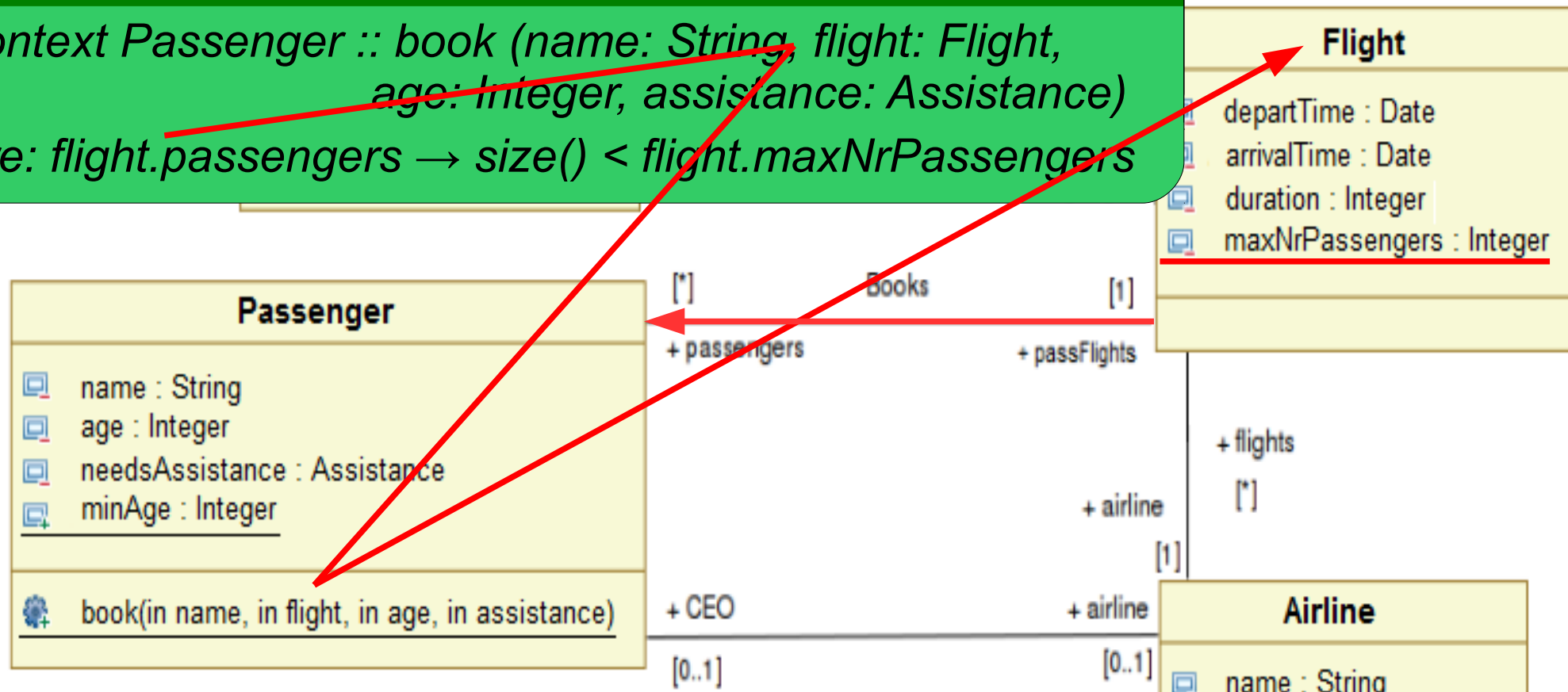
*context Passenger :: book (name: String, flight: Flight,
age: Integer, assistance: Assistance)
pre: flight.passengers $\rightarrow$ size() < flight.maxNrPassengers*



Bedeutung: ?

Beispiel

```
context Passenger :: book (name: String, flight: Flight,  
                           age: Integer, assistance: Assistance)  
pre: flight.passengers → size() < flight.maxNrPassengers
```



Bedeutung: Vor Buchung von Passagier auf Flug muss Anzahl registrierter Passagiere für diesen Flug kleiner als maximale Anzahl für den Flug sein.

Beispiel

```
context Passenger :: book (name: String, flight: Flight, age: Integer,  
                           assistance: Assistance)  
pre: flight.passengers → not exists  
    (p: Passenger | p.name = name and p.age = age)
```

Bedeutung: ?

Beispiel

```
context Passenger :: book (name: String, flight: Flight, age: Integer,  
                           assistance: Assistance)  
  
pre: flight.passengers → not exists  
    (p: Passenger | p.name = name and p.age = age)
```

Bedeutung:

Vor Ausführung von *book()*:

- existiert kein *Passenger*-Objekt, dessen Attribute *name* und *age* die Werte aus *book* enthalten.

Nachbedingung:

- Bedingung, die **am Ende einer Operationsausführung** vom Rückgabewert, finalem Objektzustand, Argumenten und initialem Objektzustand erfüllt sein muss.
 - Unter Annahme, dass Vorbedingungen erfüllt sind.
- **Spezifiziert** beabsichtigte Ergebnisse und **Zustandsänderungen** (was), aber nicht wie sie geschehen (wie).
- Kann initialen Zustand eines Objektfelds mit Postfix-Notation *@pre* referenzieren (z.B. *flight.passengers@pre*).
- Kann Rückgabewert mit Schlüsselwort *result* referenzieren.

Beispiel

```
context Passenger :: book (name: String, flight: Flight, age: Integer,  
                           assistance: Assistance)  
post: flight.passengers → size() - flight.passengers@pre → size() = 1  
and  
flight.passengers → exists (p: Passenger | p.age = age and  
p.name = name and p.needsAssistance = assistance)
```

Bedeutung: ?

Beispiel

```
context Passenger :: book (name: String, flight: Flight, age: Integer,  
                           assistance: Assistance)  
post: flight.passengers → size() - flight.passengers@pre → size() = 1  
and  
flight.passengers → exists (p: Passenger | p.age = age and  
p.name = name and p.needsAssistance = assistance)
```

Bedeutung:

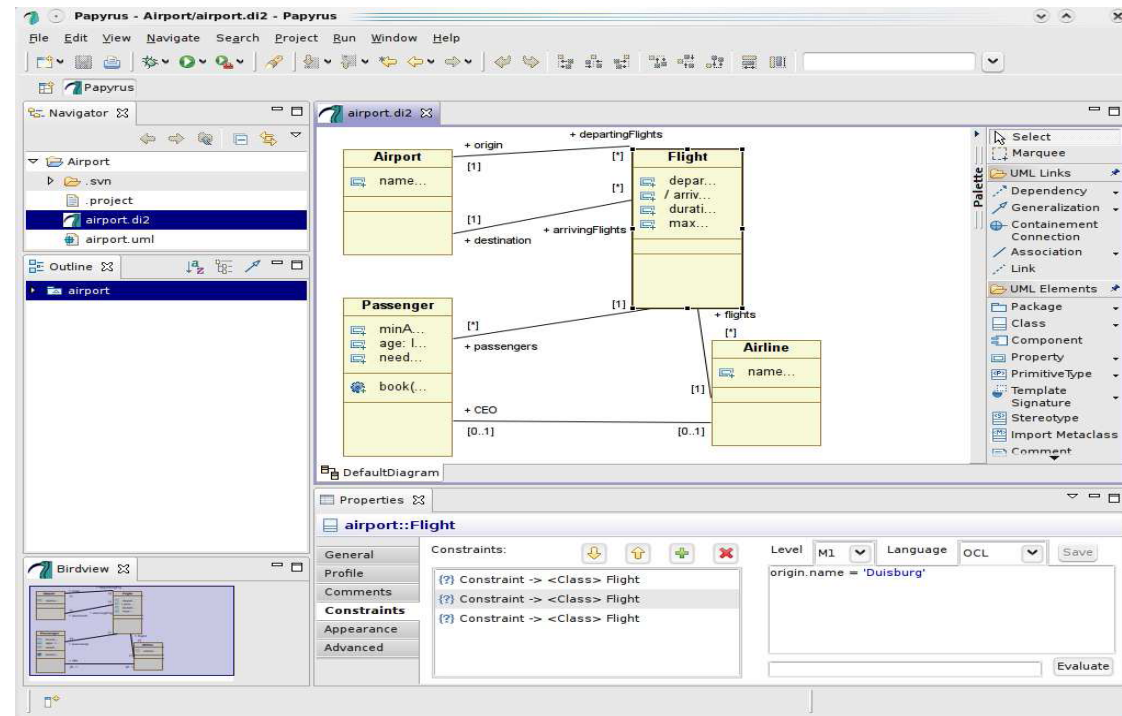
Nach Ausführung von *book()*:

- erreicht die Assoziation *passengers* ein zusätzliches Objekt und
- existiert ein *Passenger*-Objekt, dessen Attribute die Werte aus *book* enthalten.

Verschiedene UML-Editoren unterstützen Einbindung von OCL-Bedingungen.

Zum Beispiel: Papyrus.

- Frei erhältliches Open-Source Werkzeug für Modellierung mit UML 2.0, s. <http://www.papyrusuml.org/>
- Basiert auf Entwicklungsumgebung Eclipse.
- Erweiterbare Architektur von Papyrus erlaubt Hinzufügen von Diagrammen, neuen Codegeneratoren, etc.
- Erlaubt Einbinden von OCL-Bedingungen.



Logik-basierte Notation für Einschränkungen in UML-Modellen.

- **Bedingungen** an Ausführung der modellierten Systemteile formulieren und analysieren: **Invarianten, Collections, Vor- und Nachbedingungen, Wächterbedingung.**
- Erlaubt **Qualitätssicherung** der Software bereits im Design auf **Modellebene**, wo Fehler kostengünstig repariert werden können.

Vorteile:

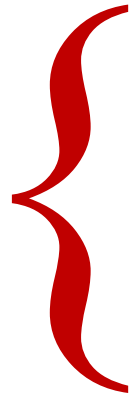
- Automatische Verifikation der Bedingung.
- **Werkzeuge** erzeugen aus OCL **Assertions** in **Java**.

Nächste Abschnitte:

Abschnitt 1.3: Petri-Netze und nebenläufige Programmierung

Weitere Informationen zum selbständigen Nachlesen / Nachschlagen.

1.1
OCL



Motivation & Einführung

Assoziationen, Navigationen, Operationen

Vor- und Nachbedingungen

Anhang

OCL-Ausdrücke: an UML-Modell gebunden.

Beschreiben Einschränkungen für Elemente des Modells, zu dem sie gehören.

Zwei Arten von **Einschränkungen** spezifizieren und verifizieren:

- **Fortlaufende** Zustandsbeschränkung (mit Invarianten).
- Zustandsbeschränkungen **vor** bzw. **nach Methodenaufruf** (mit Vor- und Nachbedingungen).

Beispiel

context Student

inv: self.MatNr >= 10000

OCL-Ausdrücke: an UML-Modell gebunden.

Beschreiben Einschränkungen für Elemente des Modells, zu dem sie gehören.

Zwei Arten von **Einschränkungen** spezifizieren und verifizieren:

- **Fortlaufende** Zustandsbeschränkung (mit Invarianten).
- Zustandsbeschränkungen **vor** bzw. **nach Methodenaufwurf** (mit Vor- und Nachbedingungen).

Beispiel

```
context Student :: nimmtTeilAn  
(VL:Vorlesung) : void  
pre: VL.FreiePlätze > 0
```

OCL-Ausdrücke: an UML-Modell gebunden.

Beschreiben Einschränkungen für Elemente des Modells, zu dem sie gehören.

Zwei Arten von **Einschränkungen** spezifizieren und verifizieren:

- **Fortlaufende** Zustandsbeschränkung (mit Invarianten).
- Zustandsbeschränkungen **vor** bzw. **nach Methodenaufruf** (mit Vor- und Nachbedingungen).

Beispiel

```
context Student :: nimmtTeilAn  
    (VL:Vorlesung):void  
post: VL.FreiePlätze =  
    VL.FreiePlätze@pre - 1
```


Folgende **Operationen** im OCL-Ausdruck benutzbar:

- **+** (**r:Real**) : **Real** : Summe von *self* und r
- **-** (**r:Real**) : **Real** : Division von *self* und r
- ***** (**r:Real**) : **Real** : Produkt von *self* und r
- **/** (**r:Real**) : **Real** : Quotient von *self* und r
- **abs()** : **Real** : Absoluter Wert von *self*
- **max** (**r : Real**) : Maximum von *self* und r
- **min** (**r : Real**) : Minimum von *self* und r
- **<** (**r : Real**) : **Boolean** : True falls *self* kleiner als r ist.
- **>** (**r : Real**) : **Boolean** : True falls *self* größer als r ist.
- ...

Folgende **Operationen** im OCL-Ausdruck benutzbar:

- **< (r : Real) : Boolean** : True falls *self* kleiner als r ist.
- **> (r : Real) : Boolean** : True falls *self größer* als r ist.
- ...

Führerschein



alter : Integer

Für einen Führerschein muss man älter als 16 Jahre alt sein.

Beispiel

context Führerschein
inv: alter > 16

Folgende Notationen sind äquivalent:

Beispiel

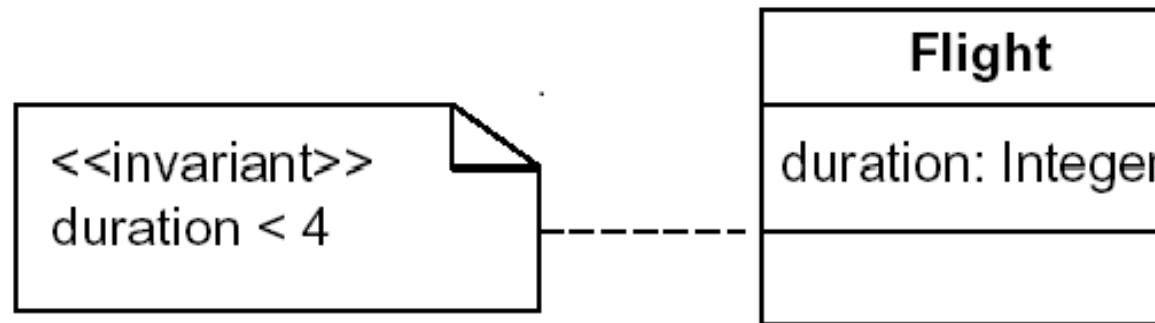
context Flug

inv: self.duration < 4

Beispiel

context Flug

inv: duration < 4



Type	Description	Values	Operators and Operations
Boolean		true, false	=, <>, and, or, xor, not, implies, if-then-else-endif (note 2)
Integer	A whole number of any size	-1, 0, 1, ...	=, <>, >, <, >=, <=, *, +, - (unary), - (binary), / (real), abs(), max(b), min(b), mod(b), div(b)
Real	A real number of any size	1.5, ...	=, <>, >, <, >=, <=, *, +, - (unary), - (binary), /, abs(), max(b), min(b), round(), floor()
String	A string of characters	'a', 'John'	=, <>, size(), concat(s2), substring(lower, upper) (1<=lower<=upper<=size), toReal(), toInteger()

Notes:

- 1) Operations indicated with parenthesis are applied with “.”, but the parenthesis may be omitted.
- 2) Example: title = (if isMale then 'Mr.' else 'Ms.' endif)

entnommen aus <http://www.di.uminho.pt/~jmf/MDSE/u2c.pdf>

- Collection von Objekten:
 - **Set:**
 - Jedes Element kommt nur einmal vor.
 - Einfaches Navigieren einer Assoziation liefert Set zurück.
 - **Bag:**
 - Gleiche Elemente dürfen mehrmals vorkommen.
 - **OrderedSet:**
 - Satz von geordneten Elementen.
 - **Sequence:**
 - Bag in dem Elemente geordnet sind.

Description	Syntax	Examples
Abstract collection of elements of type T	<code>Collection(T)</code>	
Unordered collection, no duplicates	<code>Set(T)</code>	<code>Set{1 , 2}</code>
Ordered collection, duplicates allowed	<code>Sequence(T)</code>	<code>Sequence {1, 2, 1}</code> <code>Sequence {1..4}</code> (same as <code>{1,2,3,4}</code>)
Ordered collection, no duplicates	<code>OrderedSet(T)</code>	<code>OrderedSet {2, 1}</code>
Unordered collection, duplicates allowed	<code>Bag(T)</code>	<code>Bag {1, 1, 2}</code>
Tuple (with named parts)	<code>Tuple(field1: T1, ... fieldn : Tn)</code>	<code>Tuple {age: Integer = 5, name: String = 'Joe' }</code> <code>Tuple {name = 'Joe', age = 5}</code>

Note 1: They are *value types*: “=” and “<>” compare values and not references.

Operation	Description
<code>size(): Integer</code>	The number of elements in this collection (<i>self</i>)
<code>isEmpty(): Boolean</code>	<code>size = 0</code>
<code>notEmpty(): Boolean</code>	<code>size > 0</code>
<code>includes(object: T): Boolean</code>	True if <i>object</i> is an element of <i>self</i>
<code>excludes(object: T): Boolean</code>	True if <i>object</i> is not an element of <i>self</i>
<code>count(object: T): Integer</code>	The number of occurrences of <i>object</i> in <i>self</i>
<code>includesAll(c2: Collection(T)): Boolean</code>	True if <i>self</i> contains all the elements of <i>c2</i>
<code>excludesAll(c2: Collection(T)): Boolean</code>	True if <i>self</i> contains none of the elements of <i>c2</i>
<code>sum(): T</code>	The addition of all elements in <i>self</i> (T must support "+")
<code>product(c2: Collection(T2)) : Set(Tuple(first:T, second:T2))</code>	The cartesian product operation of <i>self</i> and <i>c2</i> .

Note: Operations on collections are applied with "->" and not "."

Iterator expression	Description
iterate (iterator: T; accum: T2 = init body) : T2	Returns the final value of an accumulator that, after initialization, is updated with the value of the <i>body</i> expression for every element in the <i>source</i> collection.
exists (iterators body) : Boolean	True if <i>body</i> evaluates to true for at least one element in the <i>source</i> collection. Allows multiple iterator variables.
forAll (iterators body): Boolean	True if <i>body</i> evaluates to true for each element in the source collection. Allows multiple iterator variables.
one (iterator body): Boolean	True if there is exactly one element in the <i>source</i> collection for which <i>body</i> is true
isUnique (iterator body): Boolean	Results in true if <i>body</i> evaluates to a different value for each element in the <i>source</i> collection.
any (iterator body): T	Returns any element in the source collection for which <i>body</i> evaluates to true. The result is null if there is none.
collect (iterator body): Collection(T2)	The Collection of elements resulting from applying <i>body</i> to every member of the <i>source</i> set.

Note: The iterator variable declaration can be omitted when there is no ambiguity.

Iterator expression on Collection (T)

Softwarekonstruktion
WS 2016/17



LEHRSTUHL 14
SOFTWARE ENGINEERING

Iterator expression	Description
<code>select(iterator body): Collection(T)</code>	The Collection of elements of the <i>source</i> collection for which <i>body</i> is true. The result collection is of the same type of the <i>source</i> collection.
<code>reject(iterator body): Collection(T)</code>	The Collection of elements of the <i>source</i> collection for which <i>body</i> is false. The result collection is of the same type of the <i>source</i> collection.
<code>collectNested(iterator body): CollectionWithDuplicates(T2)</code>	The Collection of elements (allowing duplicates) that results from applying <i>body</i> (of type T2) to every member of the <i>source</i> collection. The result is not flattened. Conversions: Set -> Bag, OrderedSet -> Sequence.
<code>sortedBy(iterator body): OrderedCollection(T)</code>	Returns an ordered Collection of all the elements of the <i>source</i> collection by ascending order of the value of the <i>body</i> expression. The type T2 of the <i>body</i> expression must support "<". Conversions: Set -> OrderedSet, Bag -> Sequence.

entnommen aus <http://www.di.uminho.pt/~jmf/MDSE/u2c.pdf>

73

Operation	Description
<code>=(s: Set(T)) : Boolean</code>	Do <i>self</i> and <i>s</i> contain the same elements?
<code>union(s: Set(T)): Set(T)</code>	The union of <i>self</i> and <i>s</i> .
<code>union(b: Bag(T)): Bag(T)</code>	The union of <i>self</i> and bag <i>b</i> .
<code>intersection(s: Set(T)): Set(T)</code>	The intersection of <i>self</i> and <i>s</i> .
<code>intersection(b: Bag(T)): Set(T)</code>	The intersection of <i>self</i> and <i>b</i> .
<code>-(s: Set(T)) : Set(T)</code>	The elements of <i>self</i> , which are not in <i>s</i> .
<code>including(object: T): Set(T)</code>	The set containing all elements of <i>self</i> plus <i>object</i> .
<code>excluding(object: T): Set(T)</code>	The set containing all elements of <i>self</i> minus <i>object</i> .
<code>symmetricDifference(s: Set(T)): Set(T)</code>	The set containing all the elements that are in <i>self</i> or <i>s</i> , but not in both.

Operation	Description
flatten() : Set(T2)	If T is a collection type, the result is the set with all the elements of all the elements of <i>self</i> ; otherwise, the result is <i>self</i> .
asOrderedSet() : OrderedSet(T)	OrderedSet with elements from <i>self</i> in undefined order.
asSequence() : Sequence(T)	Sequence with elements from <i>self</i> in undefined order.
asBag() : Bag(T)	Bag will all the elements from <i>self</i> .

Operation	Description
<code>=(bag: Bag(T)) : Boolean</code>	True if <i>self</i> and <i>bag</i> contain the same elements, the same number of times.
<code>union(bag: Bag(T)): Bag(T)</code>	The union of <i>self</i> and <i>bag</i> .
<code>union(set: Set(T)): Bag(T)</code>	The union of <i>self</i> and <i>set</i> .
<code>intersection(bag: Bag(T)): Bag(T)</code>	The intersection of <i>self</i> and <i>bag</i> .
<code>intersection(set: Set(T)): Set(T)</code>	The intersection of <i>self</i> and <i>set</i> .
<code>including(object: T): Bag(T)</code>	The bag with all elements of <i>self</i> plus <i>object</i> .
<code>excluding(object: T): Bag(T)</code>	The bag with all elements of <i>self</i> without <i>object</i> .
<code>flatten() : Bag(T2)</code>	If T is a collection type: bag with all the elements of all the elements of <i>self</i> ; otherwise: <i>self</i> .
<code>asSequence(): Sequence(T)</code>	Seq. with elements from <i>self</i> in undefined order.
<code>asSet(): Set(T)</code>	Set with elements from <i>self</i> , without duplicates.
<code>asOrderedSet(): OrderedSet(T)</code>	OrderedSet with elements from <i>self</i> in undefined order, without duplicates.

Operation	Description
<code>=(s: Sequence(T)) : Boolean</code>	True if <i>self</i> contains the same elements as <i>s</i> , in the same order.
<code>union(s: Sequence(T)): Sequence(T)</code>	The sequence consisting of all elements in <i>self</i> , followed by all elements in <i>s</i> .
<code>flatten() : Sequence(T2)</code>	If <i>T</i> is a collection type, the result is the set with all the elements of all the elements of <i>self</i> ; otherwise, it's <i>self</i> .
<code>append(object: T): Sequence(T)</code>	The sequence with all elements of <i>self</i> , followed by <i>object</i> .
<code>prepend(obj: T): Sequence(T)</code>	The sequence with <i>object</i> , followed by all elements in <i>self</i> .
<code>insertAt(index : Integer, object : T) : Sequence(T)</code>	The sequence consisting of <i>self</i> with <i>object</i> inserted at position <i>index</i> ($1 \leq \text{index} \leq \text{size} + 1$)
<code>subSequence(lower : Integer, upper: Integer) : Sequence(T)</code>	The sub-sequence of <i>self</i> starting at index <i>lower</i> , up to and including index <i>upper</i> ($1 \leq \text{lower} \leq \text{upper} \leq \text{size}$)

Operation	Description
<code>at(i : Integer) : T</code>	The <i>i</i> -th element of <i>self</i> ($1 \leq i \leq \text{size}$)
<code>indexOf(object : T) : Integer</code>	The index of <i>object</i> in <i>self</i> .
<code>first() : T</code>	The first element in <i>self</i> .
<code>last() : T</code>	The last element in <i>self</i> .
<code>including(object: T): Sequence(T)</code>	The sequence containing all elements of <i>self</i> plus <i>object</i> added as last element
<code>excluding(object: T): Sequence(T)</code>	The sequence containing all elements of <i>self</i> apart from all occurrences of <i>object</i> .
<code>asBag(): Bag(T)</code>	The Bag containing all the elements from <i>self</i> , including duplicates.
<code>asSet(): Set(T)</code>	The Set containing all the elements from <i>self</i> , with duplicates removed.
<code>asOrderedSet(): OrderedSet(T)</code>	An OrderedSet that contains all the elements from <i>self</i> , in the same order, with duplicates removed.

Operation	Description
<code>append(object: T): OrderedSet(T)</code>	The set of elements, consisting of all elements of <i>self</i> , followed by <i>object</i> .
<code>prepend(object: T): OrderedSet(T)</code>	The sequence consisting of <i>object</i> , followed by all elements in <i>self</i> .
<code>insertAt(index : Integer, object : T) : OrderedSet(T)</code>	The set consisting of <i>self</i> with <i>object</i> inserted at position <i>index</i> .
<code>subOrderedSet(lower : Integer, upper : Integer) : OrderedSet(T)</code>	The sub-set of <i>self</i> starting at number <i>lower</i> , up to and including element number <i>upper</i> ($1 \leq \text{lower} \leq \text{upper} \leq \text{size}$).
<code>at(i : Integer) : T</code>	The <i>i</i> -th element of <i>self</i> ($1 \leq i \leq \text{size}$).
<code>indexOf(object : T) : Integer</code>	The index of <i>object</i> in the sequence.
<code>first() : T</code>	The first element in <i>self</i> .
<code>last() : T</code>	The last element in <i>self</i> .

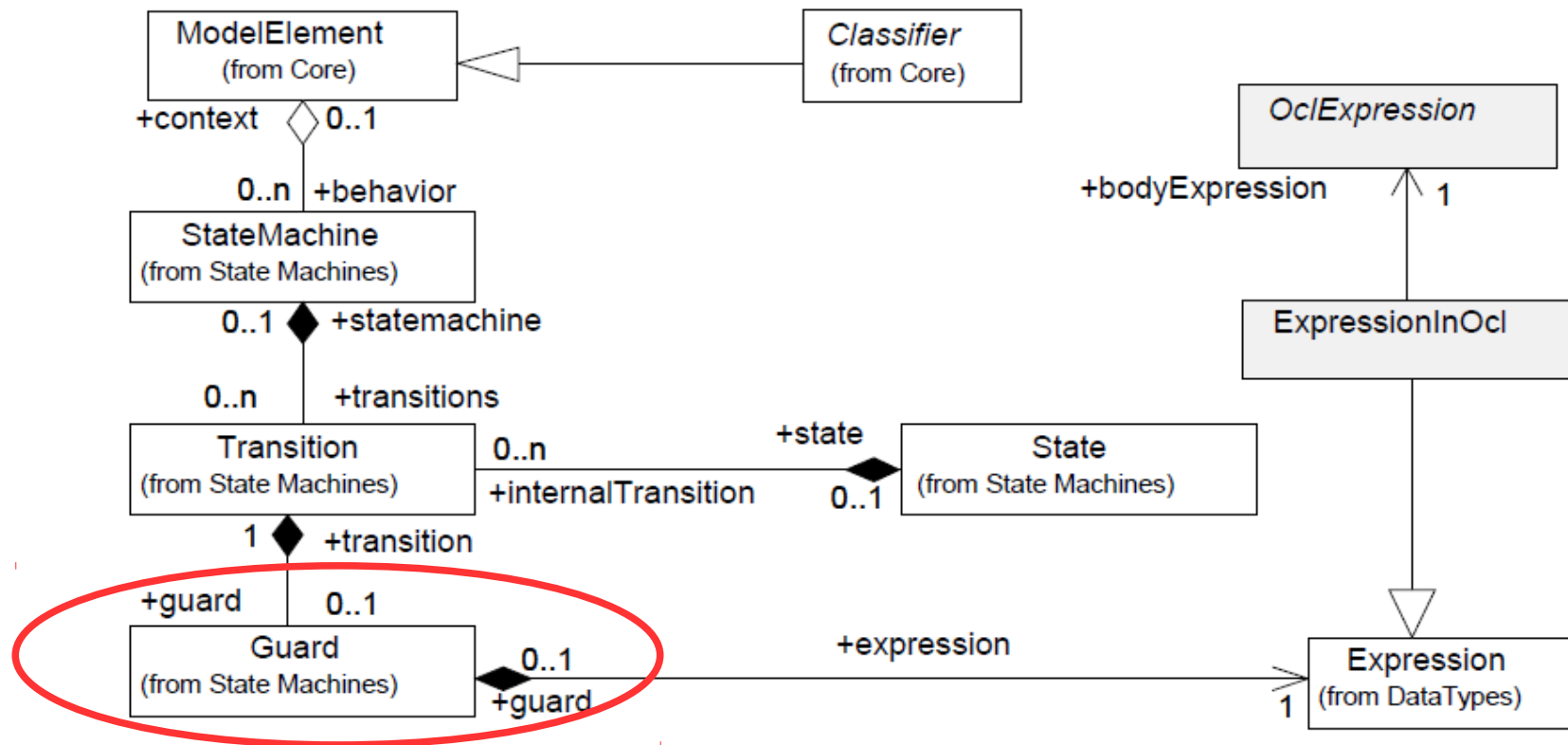
Type	Description
OclAny	Supertype for all types except for collection and tuple types. All classes in a UML model inherit all operations defined on OclAny.
OclVoid	The type OclVoid is a type that conforms to all other types. It has one single instance called <i>null</i> . Any property call applied on <i>null</i> results in <i>OclInvalid</i> , except for the operation <i>oclIsUndefined()</i> . A collection may have <i>null</i> 's.
OclInvalid	The type OclInvalid is a type that conforms to all other types. It has one single instance called <i>invalid</i> . Any property call applied on <i>invalid</i> results in <i>invalid</i> , except for the operations <i>oclIsUndefined()</i> and <i>oclIsInvalid()</i> .
OclMessage	Template type with one parameter T to be substituted by a concrete operation or signal type. Used in some postconditions that need to constrain the messages sent during the operation execution.
OclType	Meta type.

entnommen aus <http://www.di.uminho.pt/~jmf/MDSE/u2c.pdf>

Operation	Description
<code>=(object2 : OclAny) : Boolean</code>	True if <i>self</i> is the same object as <i>object2</i> .
<code><>(object2 : OclAny) : Boolean</code>	True if <i>self</i> is a different object from <i>object2</i> .
<code>oclIsNew() : Boolean</code>	Can only be used in a postcondition. True if <i>self</i> was created during the operation execution.
<code>oclAsType(t : OclType) : OclType</code>	Cast (type conversion) operation. Useful for downcast.
<code>oclIsTypeOf(t: OclType) : Boolean</code>	True if <i>self</i> is of type <i>t</i> .
<code>oclIsKindOf(t : OclType) : Boolean</code>	True if <i>self</i> is of type <i>t</i> or a subtype of <i>t</i> .
<code>oclIsInState(s : OclState) : Boolean</code>	True if <i>self</i> is in state <i>s</i> .
<code>oclIsUndefined() : Boolean</code>	True if <i>self</i> is equal to <i>null</i> or <i>invalid</i> .
<code>oclIsInvalid() : Boolean</code>	True if <i>self</i> is equal to <i>invalid</i> .
<code>allInstances() : Set(T)</code>	Static operation that returns all instances of a classifier.

Operation	Description
hasReturned() : Boolean	True if type of template parameter is an operation call, and the called operation has returned a value.
result()	Returns the result of the called operation, if type of template parameter is an operation call, and the called operation has returned a value.
isSignalSent() : Boolean	Returns true if the OclMessage represents the sending of a UML Signal.
isOperationCall() : Boolean	Returns true if the OclMessage represents the sending of a UML Operation call.
parameterName	The value of the message parameter.

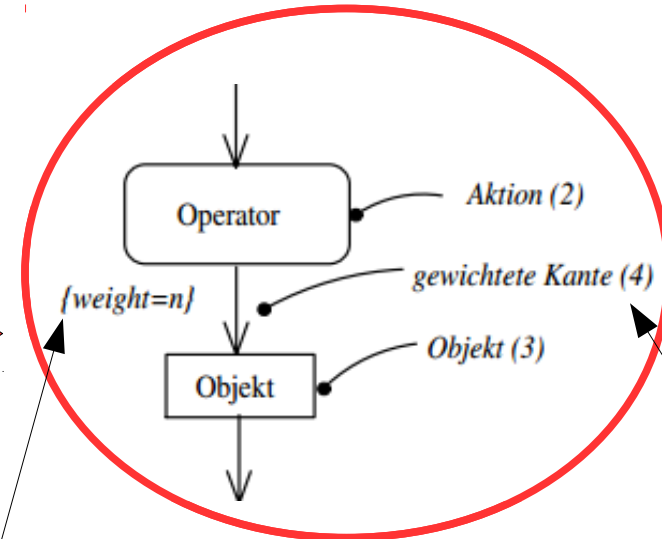
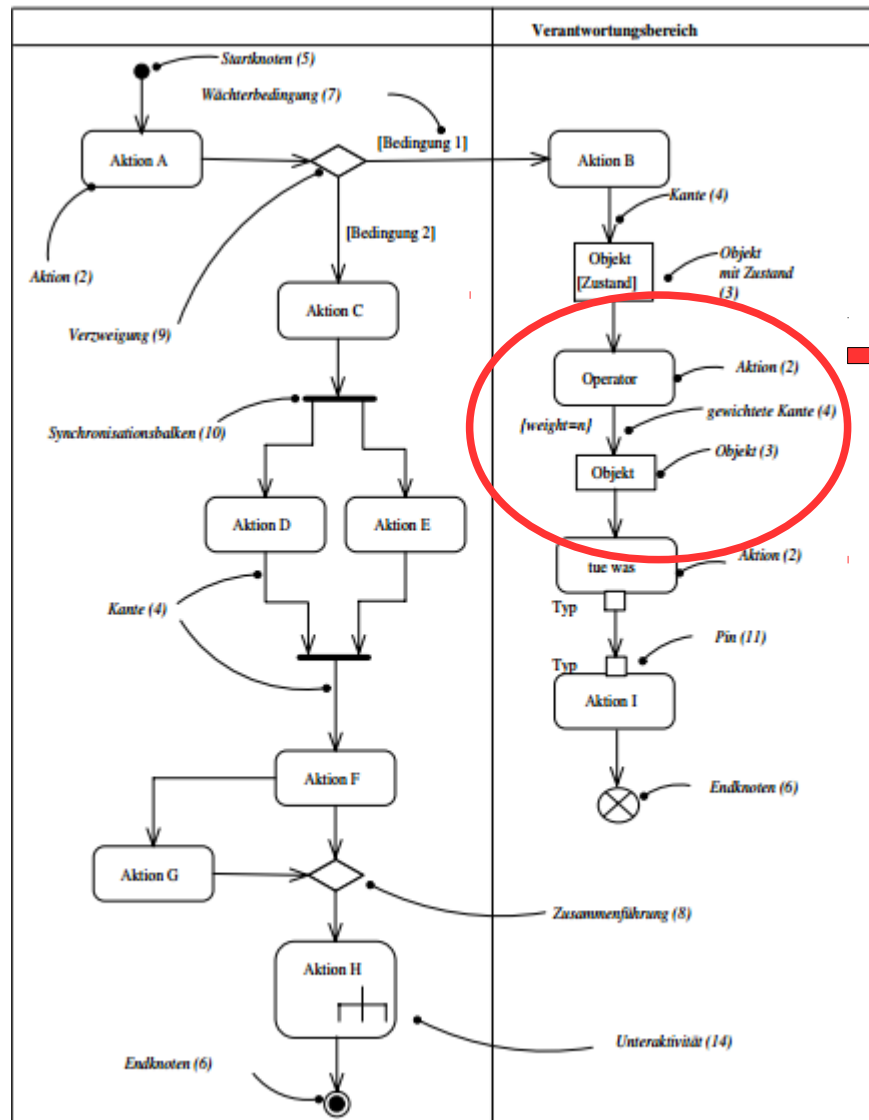
Wächterbedingung in StateCharts: Definition im UML Metamodell



Wächterbedingung (Guard condition):

- Boolesche Bedingung: validiert Auslösen eines Ereignisauftritts.

Wächterbedingung in StateCharts: Beispiel



- Kante 4: mit einer Wächterbedingung versehen
- $[weight = n]$ erfüllt $\rightarrow$ Übergang in Objekt (3)

Guard condition: Regeln

```
context ExpressionInOcl
inv: not self.guard.transition.getStateMachine().context.ocIsUndefined()
and
self.guard.transition.getStateMachine().context.ocIsKindOf(Classifier)
implies
  contextualClassifier =
    self.guard.transition.getStateMachine().context.ocAsType(Classifier)
and
self.bodyExpression.type.name = 'Boolean'
```

- A

Kontext

- B

Klassifier

1. Statemachine, in dem guard auftaucht, muss **Kontext** (A) haben, das ist ein **Klassifier** (B).
 - Kontextueller Klassifier: Besitzer des Statemachine
 - **Typ** des OCL Ausdrucks: **boolean**
2. @pre ist in einem guard nicht erlaubt.

- **Resultierende Collection** beinhaltet andere Objekte als ursprüngliche Collection.
- Wenn **Quelle Set** ist, so ist **resultierende Collection** kein Set, sondern **Bag**.
- Wenn **Quelle Sequence** oder **OrderedSet** ist, so ist neue Collection **Sequence**.

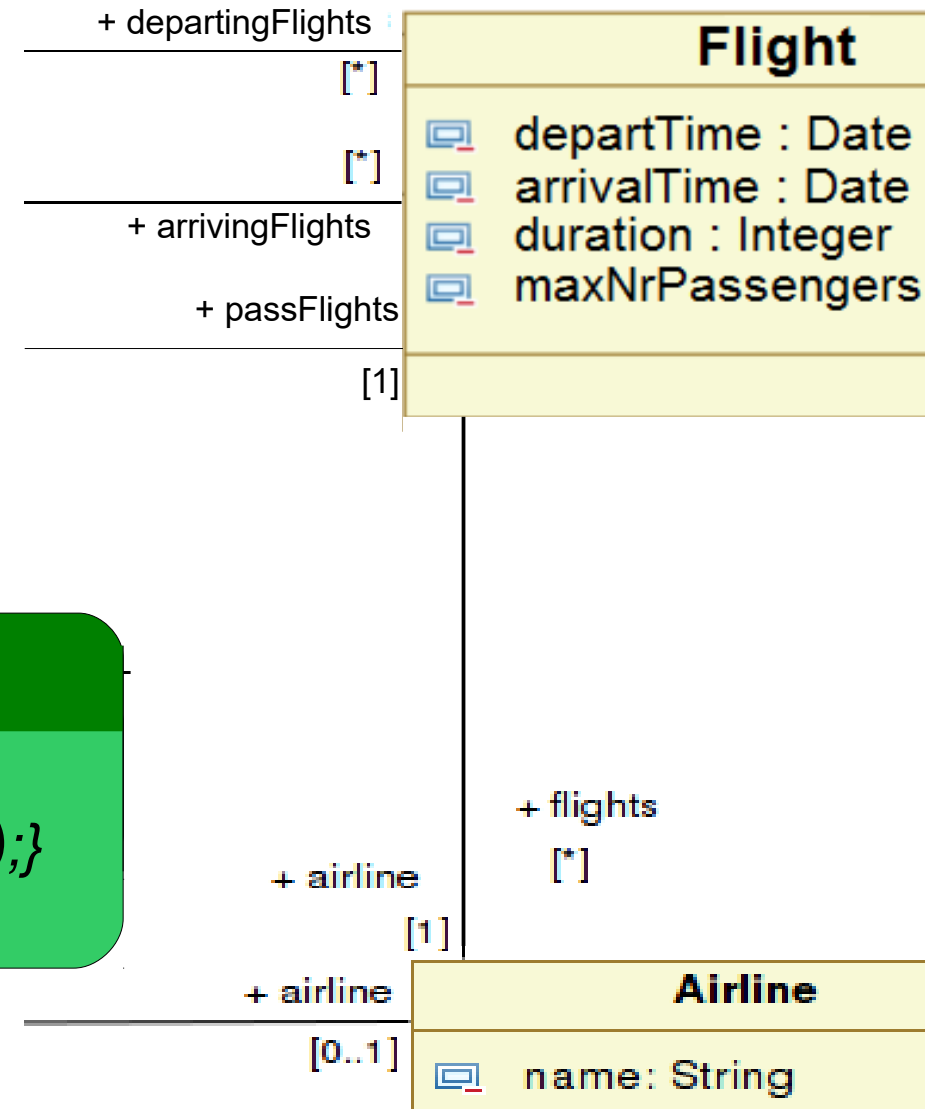
Collection-Operationen: *collect()* - Objekte

Kann benutzt werden, um neue
Collections aus Objekten am Ende
der Assoziation zu bilden, z.B.:

arrivingFlights → *collect(airline)*.

Bedeutung (in Pseudocode)

```
Collection<Airline> c = new Collection();  
foreach (f: arrivingFlights) {c.add(f.airline);}  
return c;
```



Airport	+ origin	+ departingFlights
name: String	[1]	[*]
	[1]	[*]
	+ destination	+ arrivingFlights

Flight
✏ departTime: Date ✏ / arrivalTime: Date ✏ duration: Integer ✏ maxNrPassengers: Integer

Beispiel

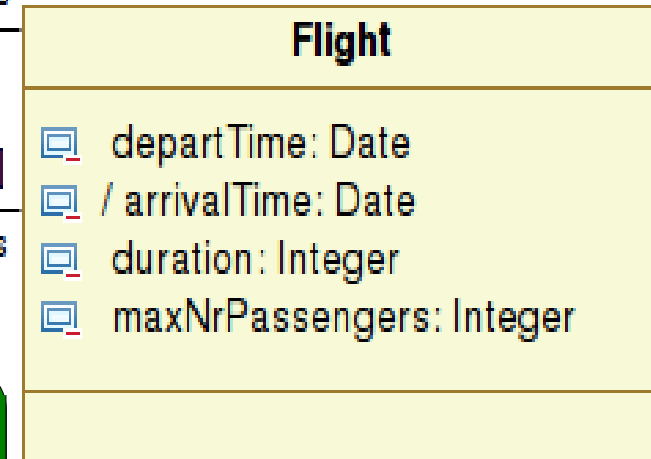
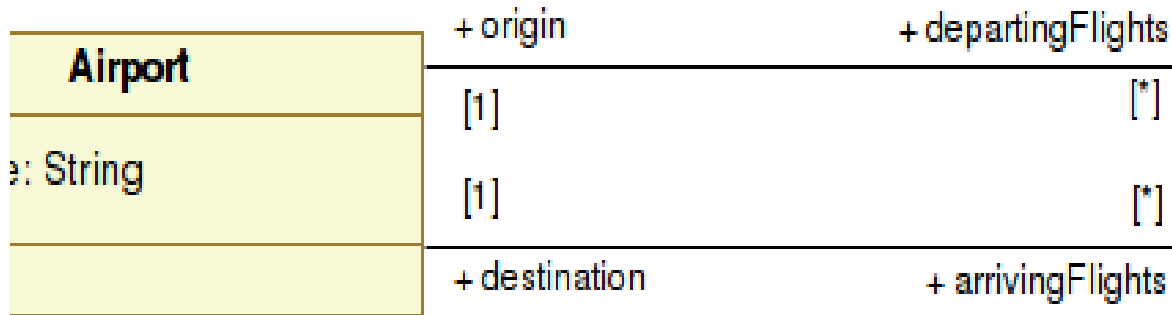
context Airport

*inv: self.arrivingFlights → size() =
self.arrivingFlights → collect(airline) → size()*

Zur Diskussion: Welche Auswirkung hätte es, wenn diese Einschränkung nicht gelten würde (z.B. wg. Code-Sharing) ?

+ flights
[*]
+ airline
[1]

Airline
✏ name: String

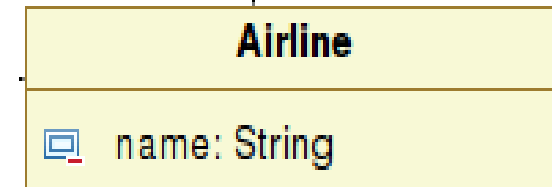
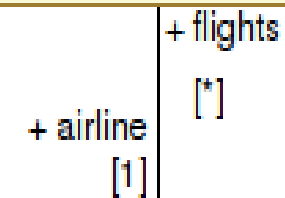


Beispiel

context Airport

*inv: self.arrivingFlights → size() =
self.arrivingFlights → collect(airline) → size()*

Antwort: Der OCL- Ausdruck könnte trotzdem gebildet und ausgewertet werden (bei Flügen mit mehr als einer Fluggesellschaft werden diese alle zu der Collection *arrivingFlights* → *collect(airline)* hinzugefügt). Er ist aber nur noch erfüllt, wenn es für jeden Flug mit >1 Fluggesellschaft entsprechend viele Flüge ohne Fluggesellschaft geben würde. Entspricht nicht der Intention!

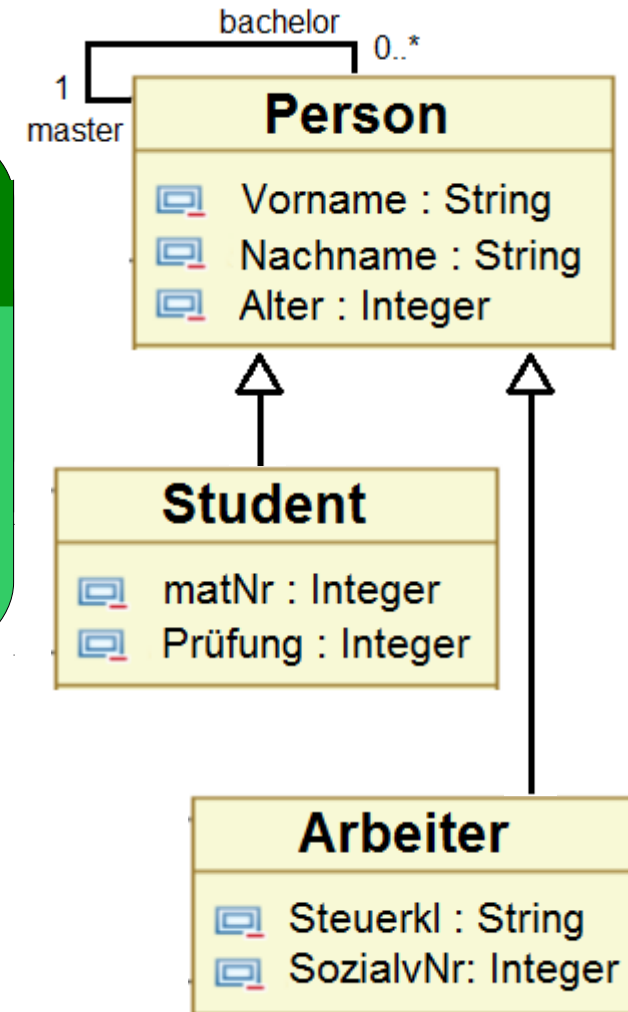


Beispiel

Context Person

```
inv: self.bachelor->select(s|  
s.isKindOf(Student))->collect(Prüfung)-> size()  
>= self.Prüfung
```

Bedeutung: ?

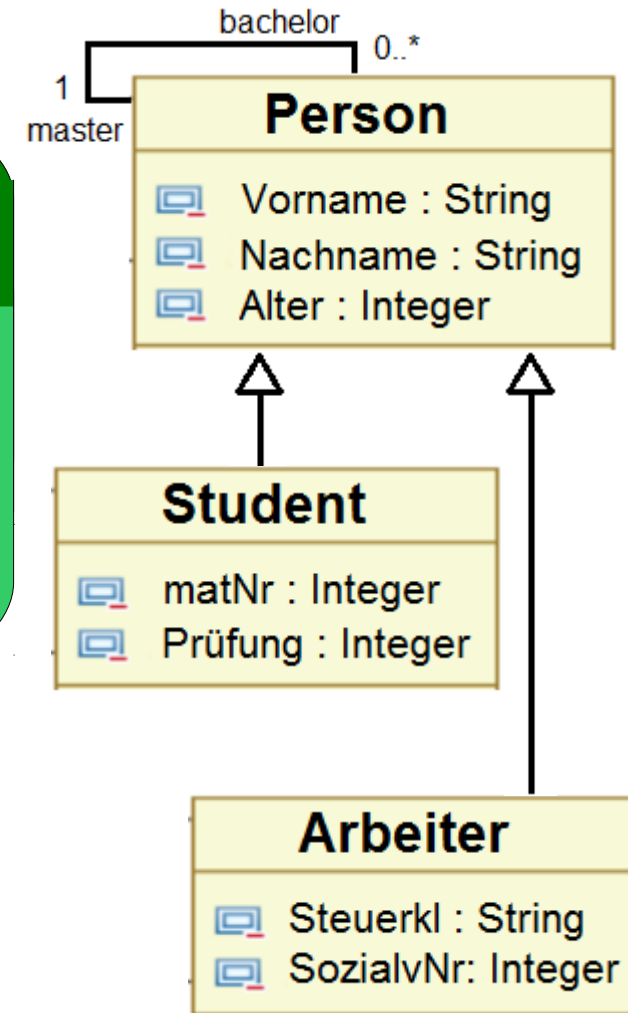


Beispiel

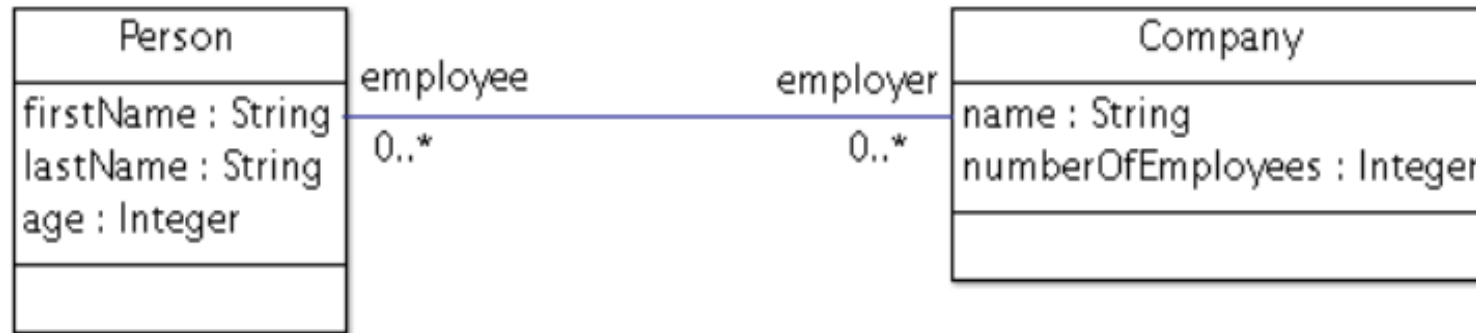
Context Person

```
inv: self.bachelor->select(s|  
s.isKindOf(Student))->collect(Prüfung)-> size()  
>= self.Prüfung
```

Bedeutung: Mehrere Bachelorstudenten
haben mehrere Prüfungen als ein
Masterstudent geschrieben.



Keiner der Mitarbeiter einer Firma darf älter als 67 sein.

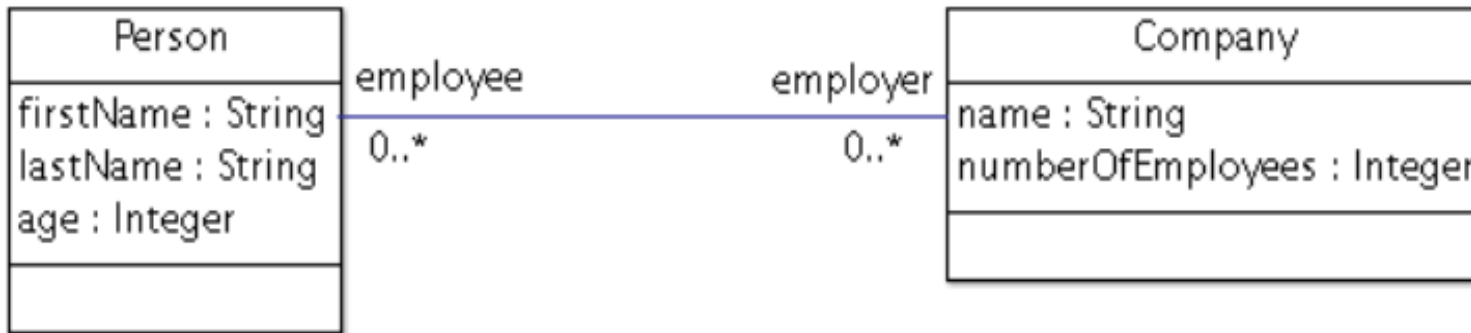


Hilfe:

Mit **isEmpty()** kann man überprüfen, ob eine Collection **leer** ist.

Lösung

Keiner der Mitarbeiter einer Firma darf älter als 67 sein.



Hilfe:

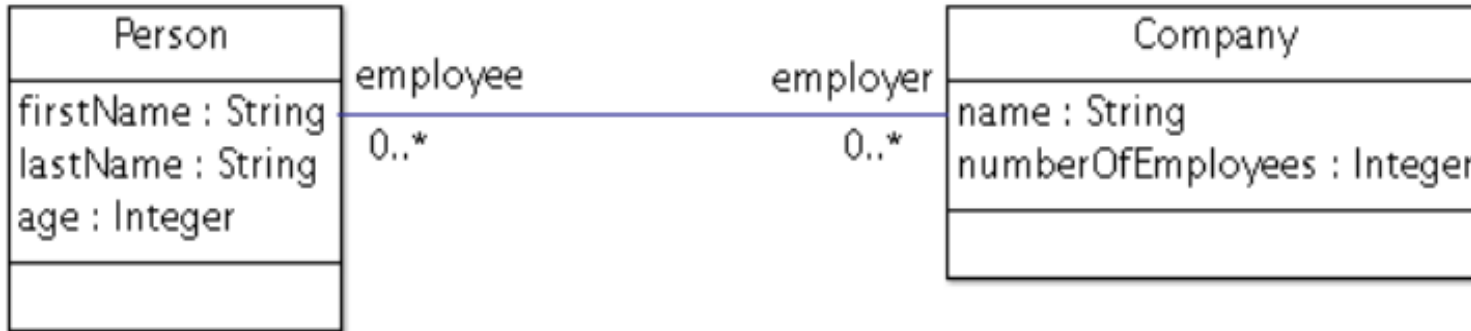
Mit **isEmpty()** kann man überprüfen, ob eine Collection **leer** ist.

Lösung

context Company

inv: self.employee → select(age > 67) → isEmpty()

Keiner der Mitarbeiter einer Firma darf älter als 67 sein.



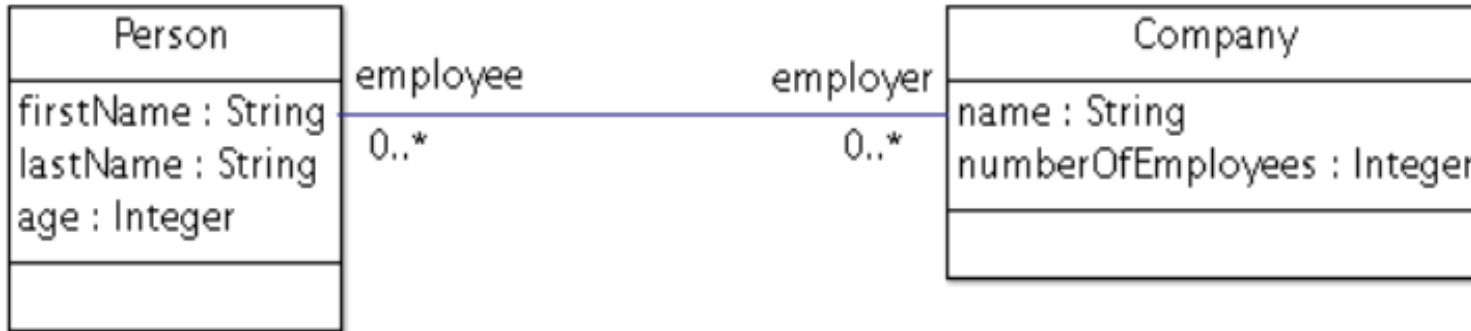
Lösung mit select()

context Company

inv: self.employee → select(age > 67) → isEmpty()

Lösung mit reject() ?

Keiner der Mitarbeiter einer Firma darf älter als 67 sein.



Lösung mit select()

context Company

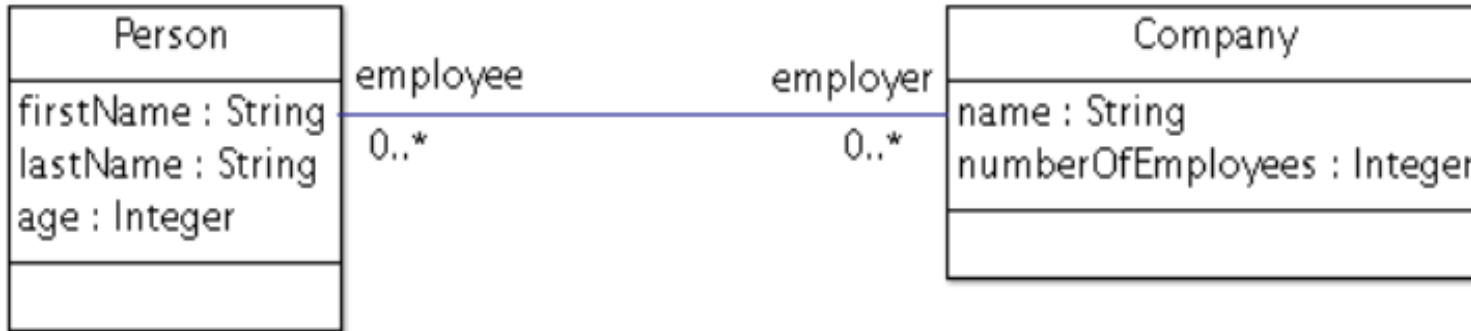
inv: self.employee → select(age > 67) → isEmpty()

Lösung mit reject() ?

context Company

inv: self.employee → reject(age <= 67) → isEmpty()

Keiner der Mitarbeiter einer Firma darf älter als 67 sein.



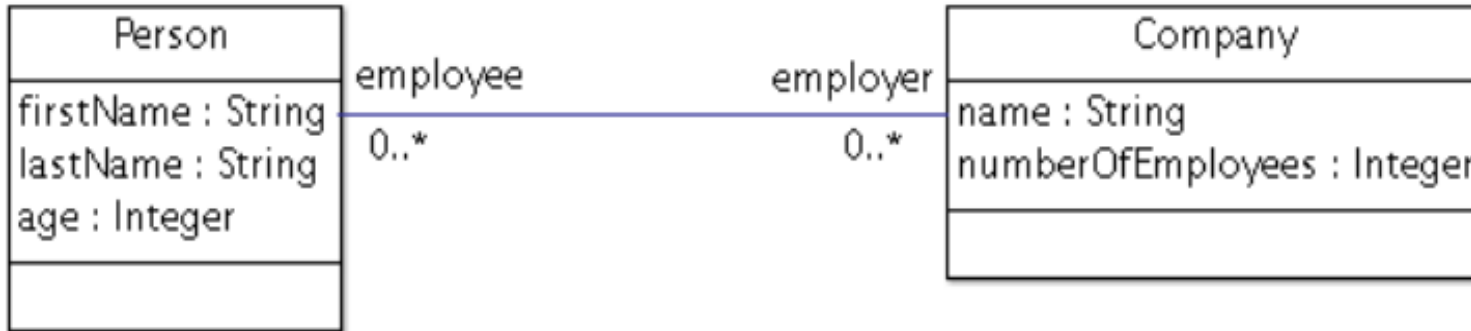
Lösung mit select()

context Company

inv: self.employee → select(age > 67) → isEmpty()

Lösung mit forAll() ?

Keiner der Mitarbeiter einer Firma darf älter als 67 sein.



Lösung mit select()

context Company

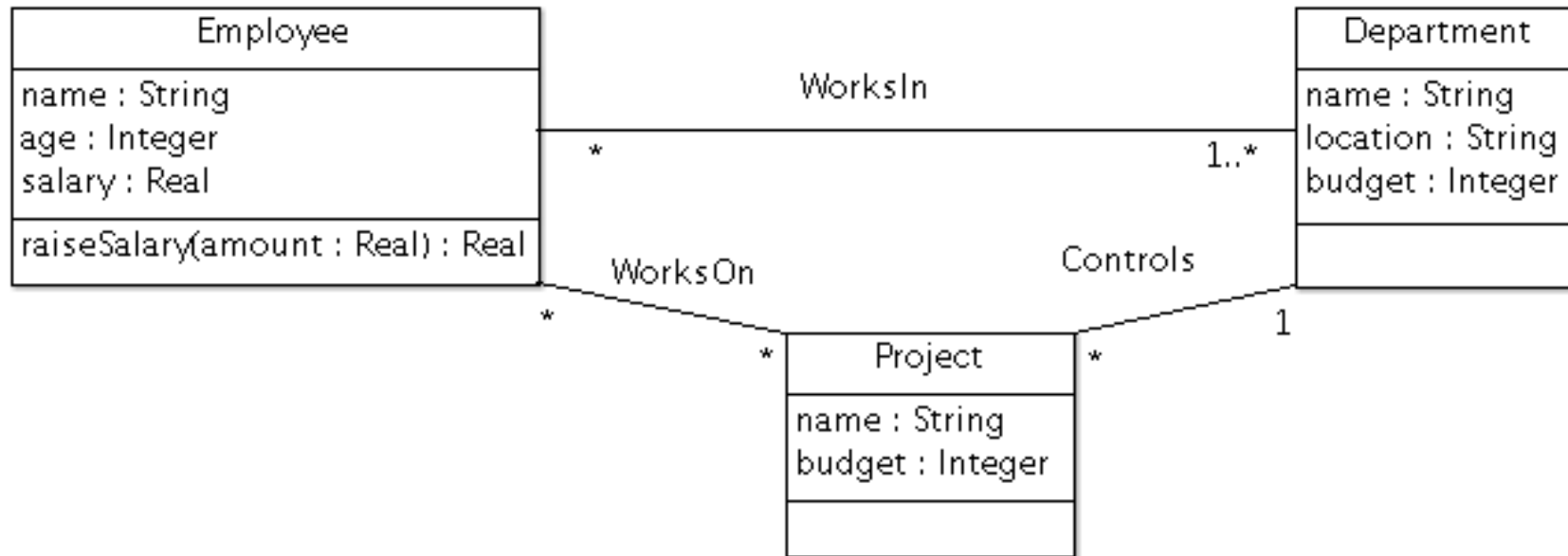
inv: self.employee → select(age > 67) → isEmpty()

Lösung mit forAll() ?

context Company

inv: self.employee → forAll(age <= 67)

Mitarbeiter, die an mehr Projekten als andere Mitarbeiter des gleichen Department arbeiten, bekommen eine höhere Vergütung.

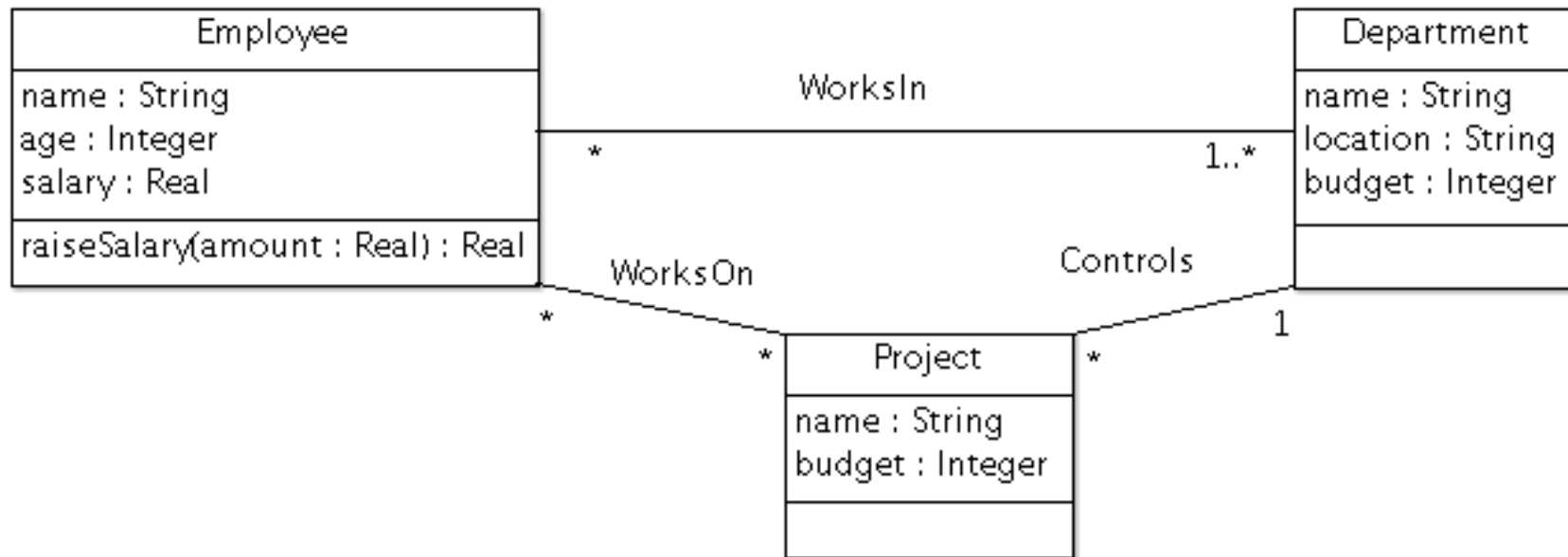


Lösung

context Department

inv: $\text{self.employee} \rightarrow \text{forAll}(e1, e2 \mid e1.\text{project} \rightarrow \text{size} > e2.\text{project} \rightarrow \text{size} \text{ implies } e1.\text{salary} > e2.\text{salary})$

Wenn der Wert (amount) positiv ist, erhöhe das Gehalt um diesen Wert.



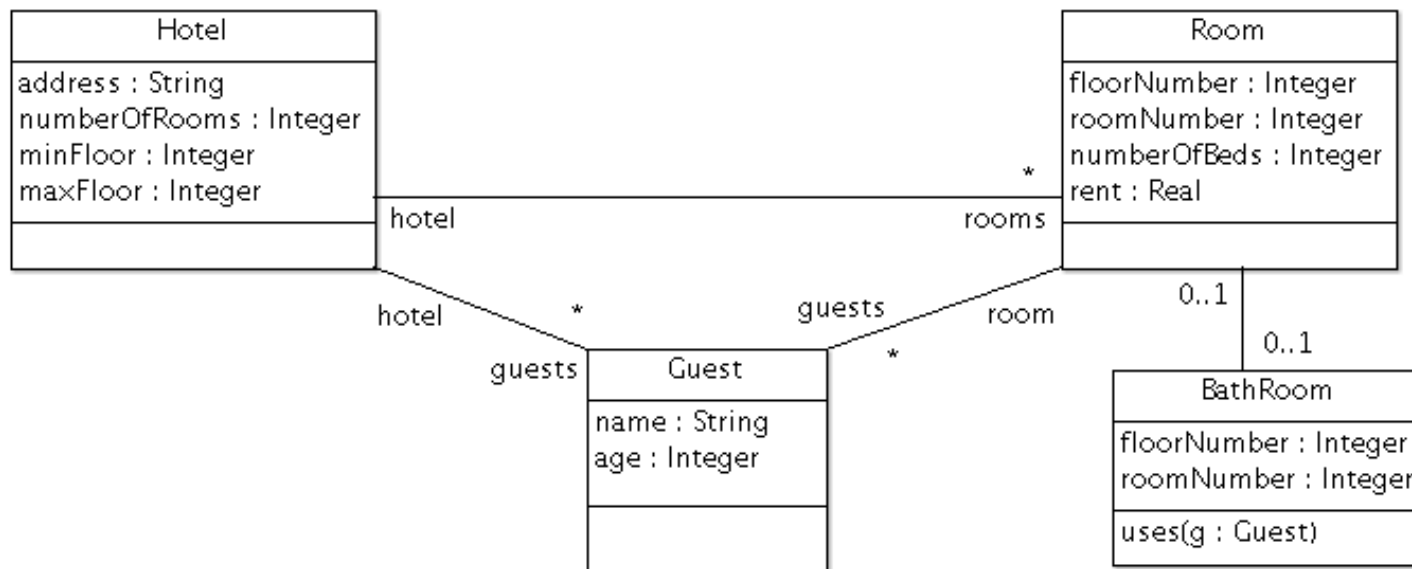
Lösung

context Employee::raiseSalary(amount: Real): Real

pre: amount > 0

post: self.salary = self.salary@pre + amount **and** result = self. salary

Die Anzahl der Gäste in einem Raum muss kleiner gleich der Anzahl der Betten sein, es sei denn einer der Gäste ist jünger als 5 Jahre, dann darf die Anzahl der Gäste einen mehr betragen.



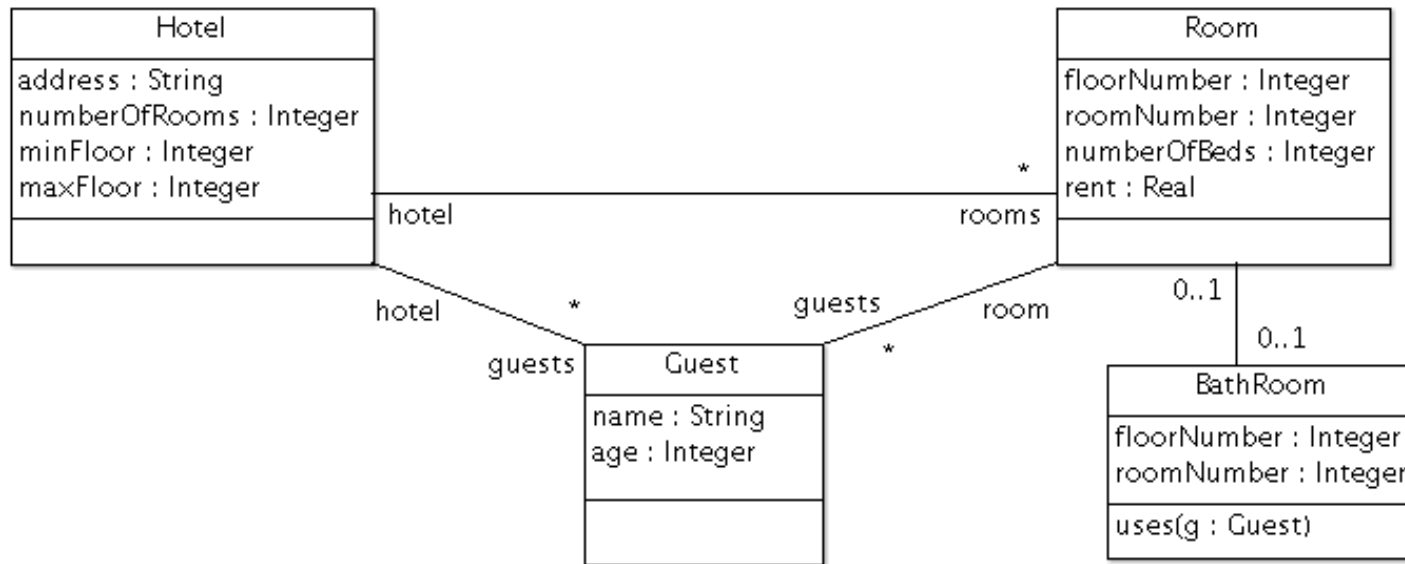
Lösung

context Room

inv: $guests \rightarrow size \leq numberOfBeds$ **or**

$(guests \rightarrow size = numberOfBeds + 1$ **and** $guests \rightarrow exists(g: Guest \mid g.age \leq 4)$

Was könnte folgender OCL-Ausdruck bedeuten?



Lösung

context Bathroom:uses(g: Guest)

pre: if room → notEmpty() **then** room.guests → includes(g)

else g.room.floorNumber = self.floorNumber **endif**

Antwort: Wenn ein Gast ein Badezimmer benutzt, dann ist es entweder das Badezimmer in seinem Raum oder ein Badezimmer auf dem selben Flur, welches keinem speziellen Raum zugeordnet ist.

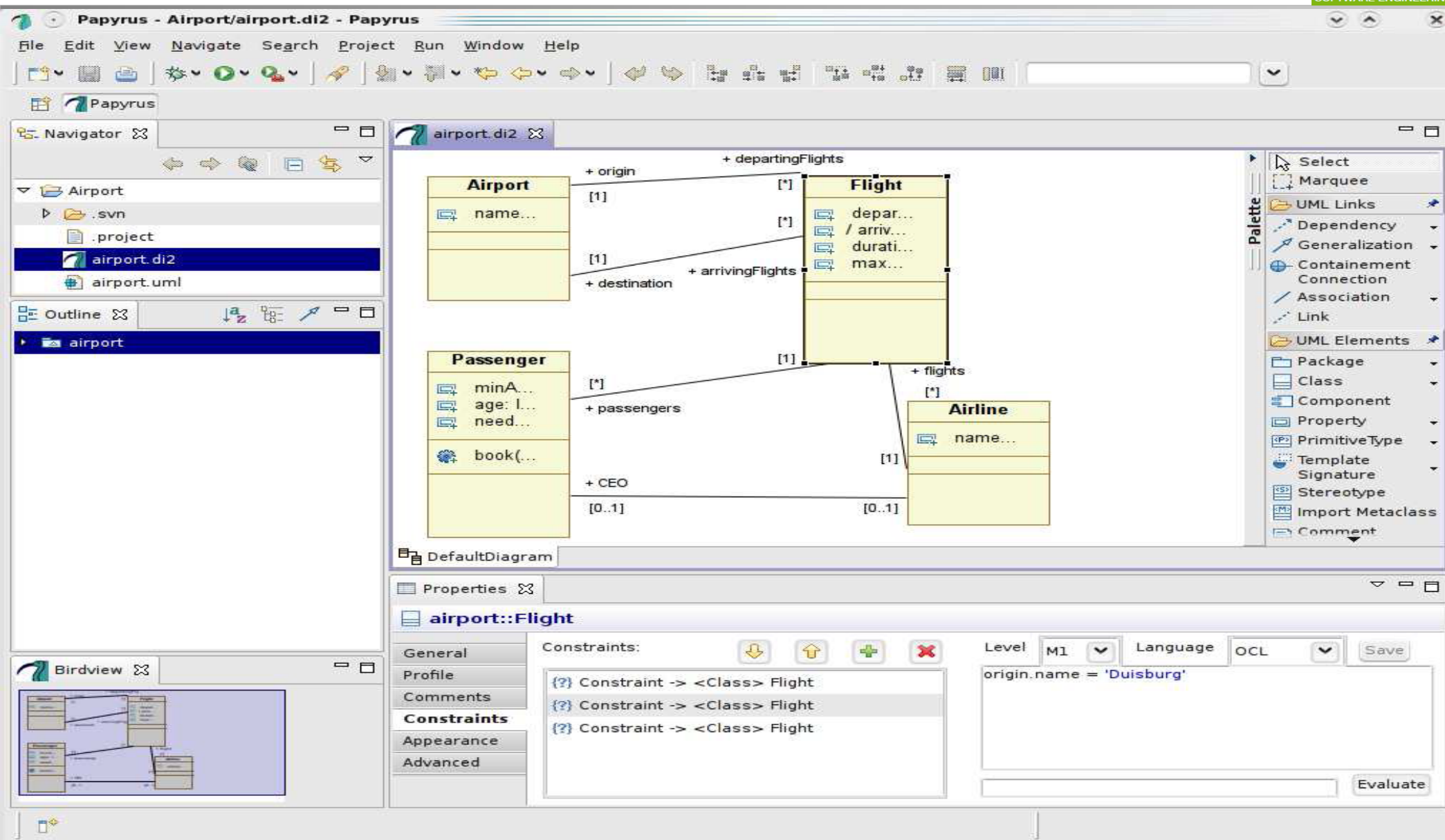
Das Einkommen einer Person darf nicht 5000 Euro überschreiten.

Person
firstName : String lastName : String age : Integer
income(Date) : Integer

Zur Erinnerung:
Schlüsselwort **result**
referenziert den
Rückgabewert.

Lösung

```
context Person::income(d: Date): Integer  
post: result <= 5000
```



- [UML09] **UML Revision Task Force**: *OMG Unified Modeling Language: Superstructure*, February 2009
<http://www.omg.org/spec/UML/2.2/.2>
- [UML10] **UML Revision Task Force**: *Object Constraint Language Specification*, February 2010
<http://www.omg.org/spec/OCL/2.2/.6>
- [WK03] **Jos Warmer and Anneke Kleppe**: *The Object Constraint Language: Getting Your Models Ready for MDA*. Addison-Wesley Longman Publishing & Co., Inc., Boston, MA, USA, 2003

Literatur:

- V. Gruhn, D. Pieper, C. Röttgers: **MDA - Effektives Software-Engineering mit UML 2 und Eclipse**. Xpert.press / Springer-Verlag, 2006.
UB e-Book: <http://www.ub.tu-dortmund.de/katalog/titel/1223129> .
 - Kapitel 3.6.
- J. Seemann, J.W. Gudenberg: **Software-Entwurf mit UML 2**. Xpert.press / Springer-Verlag, 2006.
UB e-Book: <http://www.ub.tu-dortmund.de/katalog/titel/1223020>.
 - Kapitel 14.5.
- J. Warmer, A. Kleppe: ***The Object Constraint Language: Getting Your Models Ready for MDA***. Addison-Wesley Longman Publ. & Co., Inc., 2003.
UB: <http://www.ub.tu-dortmund.de/katalog/titel/901443>
<http://www.ub.tu-dortmund.de/katalog/titel/787903>
- Object Management Group: **OCL 2.4**
<http://www.omg.org/spec/OCL/2.4/PDF>